V.I.S.O.R.

"PROTECCIÓN CON PRECISIÓN ROBÓTICA"

Manual técnico

Aplicación móvil V.I.S.O.R.

Documentación para desarrolladores · versión **0.5.0-beta** · 31 de octubre de 2026

Dato	Valor
Aplicación	V.I.S.O.R. (Vigilancia e Innovación en Sistemas Operativos Robóticos), antes «Robot Centinela»
Identificador	`com.robotcentinela.app` · `versionCode` 4 · `versionName` 0.5.0-beta
Plataforma	Android 10 o superior (`minSdk` 29), `targetSdk` 36, `compileSdk` 37
Lenguaje y UI	Kotlin 2.4.21 · Jetpack Compose (Material 3)
Backend	Supabase (Auth, PostgREST, Realtime) mediante supabase-kt 3.8.0
Código fuente	Repositorio `Robot_Centinela-Aplicacion_Movil`, rama `main`; módulos `:core` y `:app`
Documentos hermanos	Manual de usuario (`docs/manual-de-usuario/`), ADR (`docs/decisions/`), notas de versión (`docs/releases/`)

V.I.S.O.R.

"PROTECCIÓN CON PRECISIÓN ROBÓTICA"

Contenido

1. Visión general 4
   1.1 Qué es y qué no es 4
   1.2 Contexto del sistema 4
   1.3 A quién va dirigido y convenciones 5
2. Lenguajes, frameworks y herramientas 6
   2.1 Pila tecnológica 6
   2.2 Entorno de desarrollo 6
3. Arquitectura 8
   3.1 Arquitectura hexagonal en dos módulos 8
   3.2 Dominios 10
   3.3 Puertos y adaptadores 10
   3.4 Composición e inyección manual 11
   3.5 Arranque de la aplicación 12
4. Estructura de archivos 13
   4.1 Árbol del repositorio 13
   4.2 Convenciones 14
   4.3 Dónde hacer cada tipo de cambio 14
5. Conexión con el backend 15
   5.1 Configuración y validación 15
   5.2 Cliente de Supabase 15
   5.3 Autenticación y sesión 16
   5.4 Ámbito de domicilio y autorización 17
   5.5 Recursos consultados 17
   5.6 Traducción de errores 18
   5.7 Transporte y límites 19
6. Recepción de datos y actualizaciones en tiempo real 20
   6.1 Flujo de extremo a extremo 20
   6.2 Aviso en tiempo real: SupabaseTelemetryChangeNotifier 21
   6.3 Consulta de datos y reconstrucción del frame 22
   6.4 El sincronizador paso a paso 23
   6.5 Estados de sincronización 25
   6.6 Parámetros 27
   6.7 Ciclo de vida y llegada a la interfaz 27
   6.8 Historial por intervalo 28
7. Modelo de datos y diccionario de datos 30
   7.1 Contrato del frame v1.1 30
   7.2 Esquema de Supabase 31
   7.3 Correspondencia entre el frame y las tablas 37
   7.4 Tipos del dominio 37
   7.5 Catálogos 38
   7.6 Almacenamiento local 39
   7.7 Ejemplo de consulta 39

8. Reglas de dominio 39
8.1 Interpretación de lecturas 40
8.2 Frescura de los datos 41
8.3 Proyección del último dato 41
8.4 Salud de la ingesta y del robot 42
8.5 Resumen de reglas 43
9. Capa de presentación 43
9.1 Estructura de la interfaz 43
9.2 Estado y navegación 44
9.3 Tema e identidad 45
9.4 Adaptación a pantallas 46
9.5 Funciones fuera de esta versión 46
10. Compilación, variantes y distribución 47
10.1 Variantes (buildTypes) 47
10.2 Compilar y probar 48
10.3 R8 48
10.4 Firma, versionado y publicación 49
11. Pruebas y calidad 49
11.1 Estrategia 49
11.2 Fixtures 51
11.3 Pruebas en dispositivo 51
11.4 Lo que no está cubierto 51
12. Compatibilidad y limitaciones 52
12.1 Plataforma y dispositivos 52
12.2 Contrato y backend 52
12.3 Sincronización y datos 53
12.4 Funcionalidad fuera de esta versión 53
12.5 Rendimiento 53
13. Decisiones de diseño 53
14. Seguridad y privacidad 55
15. Guía de mantenimiento y extensión 56
15.1 Agregar un canal de telemetría 56
15.2 Soportar una versión nueva del contrato (schema_version 2) 56
15.3 Cambiar el nombre de una tabla o columna remota 56
15.4 Agregar un puerto y su adaptador 56
15.5 Agregar una pantalla 57
15.6 Publicar una versión 57
15.7 Problemas frecuentes de entorno 57
16. Deuda técnica y pendientes 58
Anexo A. Glosario 58
Anexo B. Mapa de la documentación del repositorio 58
Anexo C. Licencias de terceros 59

1. Visión general

1.1 Qué es y qué no es

V.I.S.O.R. es la aplicación móvil de consulta de un rover doméstico que mide gases y condiciones ambientales. Muestra en tiempo casi real las lecturas que el robot publica, su estado operativo (batería, ingesta, estado reportado), el historial y las alertas persistidas. **La aplicación es de solo lectura**: no controla el robot, no escribe telemetría ni configuración y no calcula alertas; los umbrales, la histéresis y la generación de alertas viven en la base de datos.

- **Cliente delgado sobre Supabase.** Toda la información llega por PostgREST (consultas), Realtime (avisos) y Auth (sesión).
- **Semántica antes que estética.** El dominio decide qué se puede afirmar de una lectura (señal cruda ≠ ppm, `null` ≠ 0, silencio ≠ aire seguro) antes de que la interfaz la dibuje.
- **Sin estado persistente de negocio en el teléfono.** Solo se guardan el tema y el último correo usado; no hay base de datos local ni caché de telemetría.

1.2 Contexto del sistema

La app es uno de los consumidores del proyecto de Supabase. La captura y la ingesta quedan fuera de este repositorio: el robot publica un frame por intervalo, la ingesta lo valida contra el contrato v1 y lo guarda de forma normalizada. Cada inserción confirmada es la señal que la app usa para pedir datos nuevos.

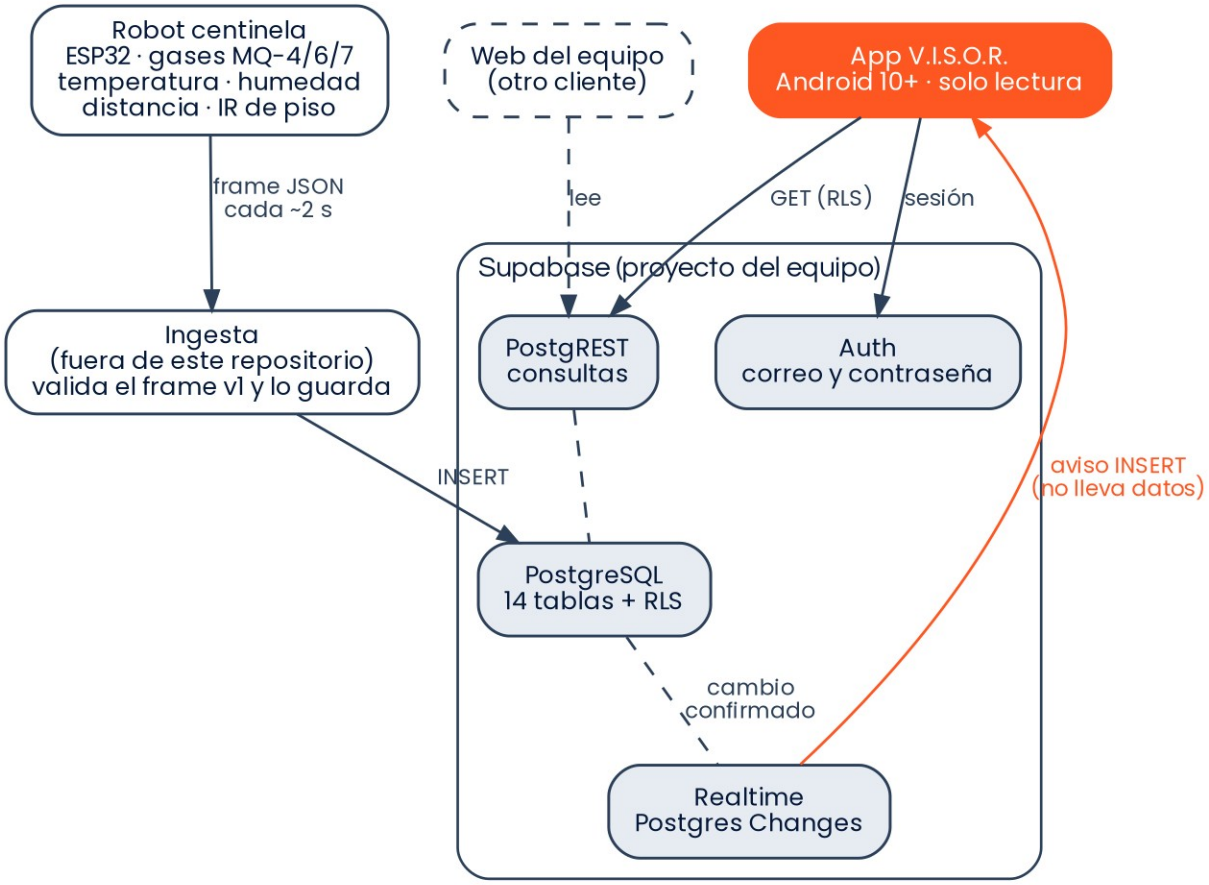


Figura 1. Contexto del sistema. El aviso de Realtime (naranja) no transporta datos: solo dispara una consulta.

1.3 A quién va dirigido y convenciones

Este manual está escrito para quien vaya a **compilar, depurar, extender o mantener** la aplicación. Asume conocimientos de Kotlin, corrutinas y Android; no repite documentación del SDK. Las convenciones son:

- Las rutas son relativas a la raíz del repositorio; en los listados, `ruta:línea` indica el origen exacto del fragmento. Los fragmentos están tomados del código de la versión 0.5.0-beta y pueden abreviarse con `// …`.
- `ADR-00x` remite a `docs/decisions/`; los códigos `B##` (backend), `R##` (producto) e `I#` (IoT) remiten a `docs/pendientes.md`.
- Las URL, claves y correos de ejemplo son **ficticios**. Nunca se documentan ni se versionan valores reales del proyecto.

2. Lenguajes, frameworks y herramientas

2.1 Pila tecnológica

Todo el código de la aplicación está escrito en **Kotlin**; no hay Java propio ni código nativo. Las versiones salen del catálogo `gradle/libs.versions.toml` y se eligieron para ser compatibles entre sí: supabase-kt 3.8.0 se publica con Ktor 3.5.1, coroutines 1.11.0, serialization 1.11.0 y kotlin-stdlib 2.4.0.

Componente	Versión	Para qué se usa
Kotlin	2.4.21	Único lenguaje. Plugins `kotlin-jvm` (`:core`), `kotlin-serialization` y `kotlin-compose` (`:app`)
Android Gradle Plugin	9.4.1	Compilación, R8, empaquetado y firma
Gradle	9.6.0 (wrapper)	Orquestación; `org.gradle.jvmargs=-Xmx2048m`, caché de compilación activa
JDK	21	Toolchain de `:core` y `compileOptions` de `:app`
Jetpack Compose	BOM 2026.09.00	`ui`, `foundation`, `material3`, `ui-graphics`; interfaz declarativa
Activity Compose · Lifecycle	1.13.0 · 2.11.0	`ComponentActivity`, `viewModel()`, `collectAsStateWithLifecycle`, `LifecycleStartEffect`
Navigation Compose	2.10.2	Rutas tipadas (`@Serializable data object`) en `MainScreen`
core-splashscreen	1.0.1	Pantalla de inicio de la marca (`Theme.Centinela.Starting`)
supabase-kt	3.8.0 (BOM)	`auth-kt`, `postgrest-kt`, `realtime-kt`; cliente del backend
Ktor (motor OkHttp)	3.5.1	Transporte HTTP/WebSocket de supabase-kt
kotlinx.coroutines	1.11.0	`Flow`, `StateFlow`, `Channel`; sincronización y estado de UI
kotlinx.serialization	1.11.0	DTO de los adaptadores y rutas de navegación
JUnit Jupiter · kotlin-test	BOM 6.1.3	Pruebas unitarias en JVM (`useJUnitPlatform()`); `kotlinx-coroutines-test`
Poppins · Questrial	OFL 1.1	Cuerpo e interfaces · titulares (`res/font`, licencias en `app/OFL-*.txt`)

Quedan fuera, de forma deliberada: Room/DataStore (no hay base local), Hilt/Koin (inyección manual, ADR-002), Retrofit (el acceso HTTP lo resuelve supabase-kt), WorkManager y notificaciones push.

2.2 Entorno de desarrollo

- **JDK 21** (por ejemplo Temurin 21) y **Android SDK** con la plataforma API 37 y build-tools actuales. `sdk.dir` va en `local.properties`.
- **Memoria:** el heap de Gradle está fijado en 2 GB (`org.gradle.jvmargs`); en equipos con poca RAM conviene `--no-daemon --max-workers=1` y ejecutar una tarea de Gradle a la vez (una compilación con R8 tarda varios minutos).

- **Dispositivo:** las pruebas de interfaz se hacen en teléfono físico por ADB; el proyecto no usa emulador.

Listado 1 · Configuración local (valores ficticios)

```
# local.properties  (no se versiona; hay una plantilla en local.properties.example)
sdk.dir=/ruta/al/Android/Sdk
supabase.url=https://tu-proyecto.supabase.co
supabase.publishableKey=sb_publishable_valor_ficticio
# Solo builds debug: autocompleta el formulario de inicio de sesión
centinela.devEmail=prueba@ejemplo.com
centinela.devPassword=contraseña_de_prueba
```

Importante

Solo se acepta la clave **pública** (`anon` o `sb_publishable_…`). La app rechaza al arrancar cualquier clave `service_role` o `sb_secret_…` (sección 5.1).

3. Arquitectura

3.1 Arquitectura hexagonal en dos módulos

El proyecto aplica **arquitectura hexagonal (puertos y adaptadores)** con organización por dominio de negocio (*screaming architecture*). La separación se materializa en dos módulos de Gradle, de modo que el compilador —y no solo la disciplina del equipo— impide que el dominio dependa de Android o de Supabase (ADR-001).

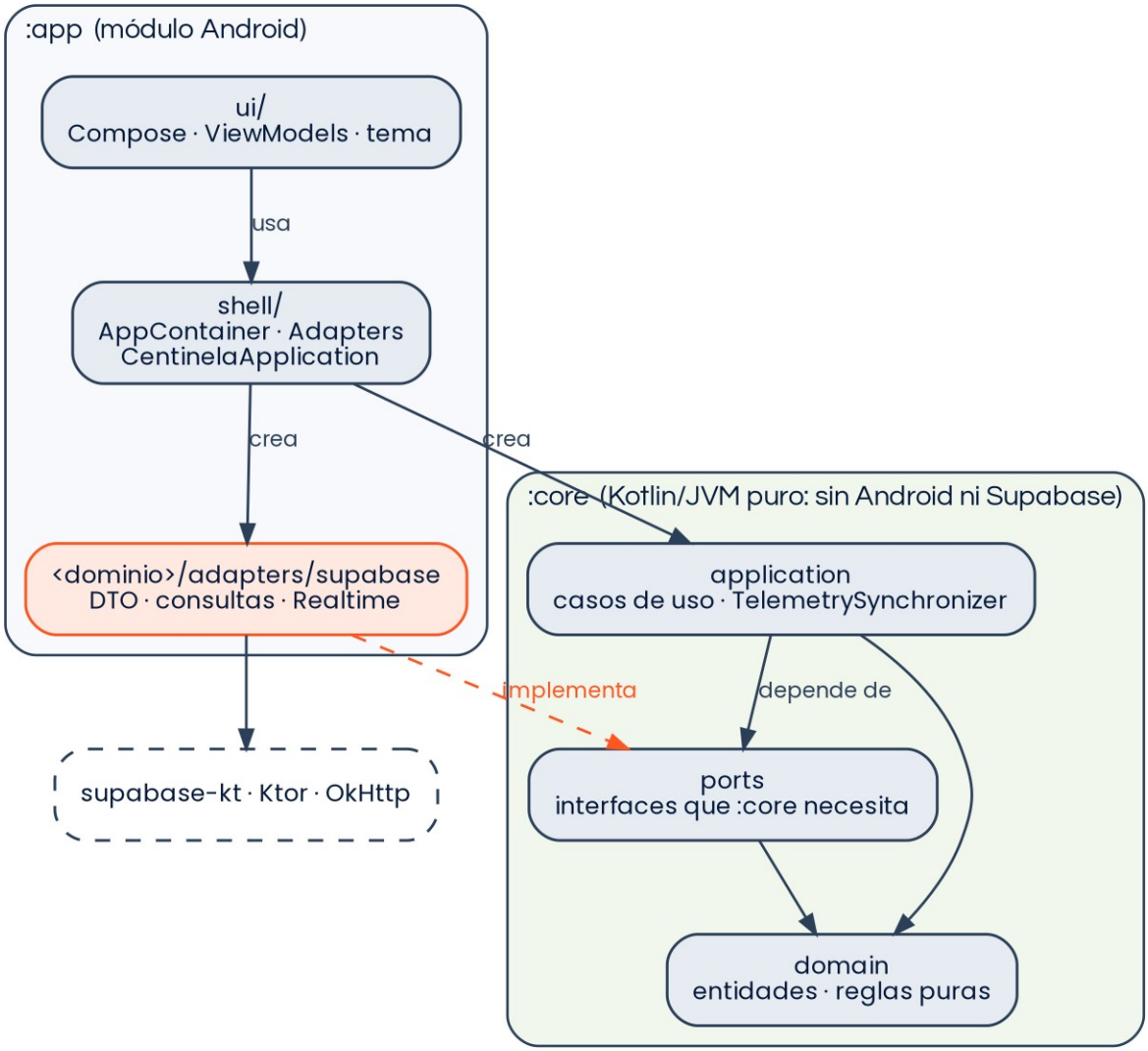


Figura 2. Módulos y capas. Las flechas apuntan hacia el dominio; los adaptadores implementan los puertos.

Capa	Dónde	Contiene	Puede depender de
`domain`	:core · `<dominio>/domain`	Entidades, objetos de valor y reglas puras (`ReadingInterpreter`, `FreshnessPolicy`, `LatestTelemetryProjection`)	Biblioteca estándar y `java.time`
`ports`	:core · `<dominio>/ports`	Interfaces que el núcleo necesita del exterior (`TelemetryQueryPort`, `AuthPort`)	domain
`application`	:core · `<dominio>/application`	Casos de uso y coordinación (`TelemetrySynchronizer`, `SelectHomeScope`)	domain, ports, coroutines
`adapters`	:app · `<dominio>/adapters/supabase`	Implementación de los puertos con supabase-kt, DTO y mapeadores	Todo lo anterior y el SDK
`shell`	:app · `shell/`	`AppContainer`, `Adapters`, `CentinelaApplication`, configuración de Supabase	Todo
`ui`	:app · `ui/`	Pantallas Compose, ViewModels, modelos de presentación y tema	application (vía `AppContainer`) y domain

`:core` solo declara `kotlinx-coroutines-core`. Una comprobación rápida de que la regla se mantiene:

Listado 2 · Verificación de la regla de dependencias

```
grep -rE "import (android|io\.github\.jan|io\.ktor)" core/   # no debe devolver nada
```

Listado 3 · Declaración de módulos `settings.gradle.kts:19`

```
// :core -> dominio, casos de uso y puertos (Kotlin/JVM puro, sin Android ni Supabase).
// :app  -> adaptadores (Supabase) y raíz de composición Android. Depende de :core, nunca al revés.
include(":core", ":app")
```

3.2 Dominios

Paquete	Responsabilidad	Tipos principales
`identity`	Sesión, membresías y ámbito de domicilio	`AuthPort`, `HomeScopePort`, `CurrentHomeScope`, `HomeScope`, `SessionState`
`telemetry`	Frame, cursor `stream_version`, proyección del último dato, sincronización e historial	`TelemetryFrame`, `FrameRecord`, `LatestTelemetryProjection`, `TelemetrySynchronizer`
`environmentalmonitoring`	Interpretación de lecturas, frescura, capacidades de canal	`ReadingInterpreter`, `FreshnessPolicy`, `DescribeChannelStatus`, `ChannelCapability`
`robothealth`	Batería, estado reportado e ingesta	`BatteryInterpretation`, `IngestConnectivityPolicy`, `EvaluateRobotHealth`
`incidents`	Alertas persistidas (solo lectura)	`Alert`, `AlertState`, `AlertQueryPort`, `QueryAlertHistory`
`missions`	Misiones y su resumen	`MissionSummary`, `MissionQueryPort`, `ListMissions`
`homemapping`	Pose e identificadores de mapa (el mapa 2D no está en esta versión, ADR-009)	`Pose`, `MapVersionId`, `StationId`
`shared`	Errores y tipos transversales	`CentinelaException`, `PageRequest`, `UtcInterval`

3.3 Puertos y adaptadores

Puerto (:core)	Adaptador (:app)	Recurso de Supabase	Estado
`AuthPort`	`SupabaseAuthAdapter`	Auth (correo y contraseña)	Implementado
`HomeScopePort`	`SupabaseHomeScopeAdapter`	`memberships` → `homes`, `robots`	Implementado
`TelemetryQueryPort`	`SupabaseTelemetryQueryAdapter`	`telemetry_frames` + 6 tablas hijas	Implementado
`TelemetryChangeNotifier`	`SupabaseTelemetryChangeNotifier`	Realtime sobre `telemetry_frames`	Implementado
`AlertQueryPort`	`SupabaseAlertQueryAdapter`	`alerts`	Implementado
`CapabilitiesPort`	`SupabaseCapabilitiesAdapter`	`channels` (clave, tipo, unidad)	Parcial: sin gas objetivo ni periodo
`MissionQueryPort`	`SupabaseMissionQueryAdapter`	`missions`	Parcial: sin estaciones ni energía
`MetricAggregatePort`	—	`metric_aggregates` (sin contrato)	Pendiente (`aggregates = null`)

3.4 Composición e inyección manual

No hay framework de inyección (ADR-002). `AppContainer` es la **raíz de composición**: recibe los adaptadores ya construidos, instancia políticas y casos de uso y expone a la interfaz solo lo que necesita. Los puertos sin contrato real (`aggregates`) se mantienen como `null` para que la UI muestre «pendiente» en lugar de inventar datos.

Listado 4 · AppContainer: adaptadores, políticas y casos de uso
`app/src/main/kotlin/com/robotcentinela/app/shell/AppContainer.kt:42`

```
// --- Adaptadores ---
val authPort: AuthPort = adapters.auth
private val telemetryQuery = adapters.telemetryQuery
private val telemetryNotifier = adapters.telemetryNotifier

// --- Estado compartido de la sesión ---
val currentHomeScope = CurrentHomeScope()

// --- Políticas con sus valores iniciales documentados (ADR-004) ---
val freshnessPolicy = FreshnessPolicy()
private val ingestConnectivityPolicy = IngestConnectivityPolicy()

// --- Casos de uso ---
val signIn = SignIn(authPort)
val signOut = SignOut(authPort, currentHomeScope)
val recoverAccess = RecoverAccess(authPort)
val listAuthorizedHomes = ListAuthorizedHomes(adapters.homeScope)
val selectHomeScope = SelectHomeScope(adapters.homeScope, currentHomeScope)
val queryTelemetryHistory = QueryTelemetryHistory(telemetryQuery, currentHomeScope)
val queryAlertHistory = QueryAlertHistory(adapters.alertQuery, currentHomeScope)
val describeChannelStatus = DescribeChannelStatus(freshnessPolicy, clock)
val evaluateRobotHealth = EvaluateRobotHealth(ingestConnectivityPolicy, freshnessPolicy, clock)
```

Listado 5 · Un sincronizador por robot, validado contra el ámbito de domicilio
`app/src/main/kotlin/com/robotcentinela/app/shell/AppContainer.kt:76`

```
fun telemetrySynchronizer(robotId: RobotId, config: SyncConfig = SyncConfig()): TelemetrySynchronizer {
    currentHomeScope.requireRobot(robotId)
    return TelemetrySynchronizer(robotId, telemetryQuery, telemetryNotifier, config)
}
```

Listado 6 · Adapters.supabase: único lugar donde se elige la implementación de cada puerto
`app/src/main/kotlin/com/robotcentinela/app/shell/Adapters.kt:55`

```
fun supabase(config: SupabaseConfig, scope: CoroutineScope, publicRead: Boolean = false): Adapters {
    val client = SupabaseClientFactory.create(config)
    return Adapters(
        auth = if (publicRead) PublicReadAuthAdapter() else SupabaseAuthAdapter(client, scope),
        homeScope = if (publicRead) PublicReadHomeScopeAdapter(client) else
            SupabaseHomeScopeAdapter(client),
        telemetryQuery = SupabaseTelemetryQueryAdapter(client, publicRead),
        telemetryNotifier = SupabaseTelemetryChangeNotifier(client),
        alertQuery = SupabaseAlertQueryAdapter(client),
        capabilities = SupabaseCapabilitiesAdapter(client),
        missions = SupabaseMissionQueryAdapter(client),
    )
}
```

3.5 Arranque de la aplicación

`CentinelaApplication` crea el contenedor de forma **diferida** y guarda el resultado en un `Result`: si `local.properties` no contiene una configuración válida, la interfaz muestra la causa en lugar de cerrarse. `RootViewModel` combina el estado de la sesión, el domicilio seleccionado y un indicador de «elegir otro domicilio» para decidir qué pantalla se dibuja.

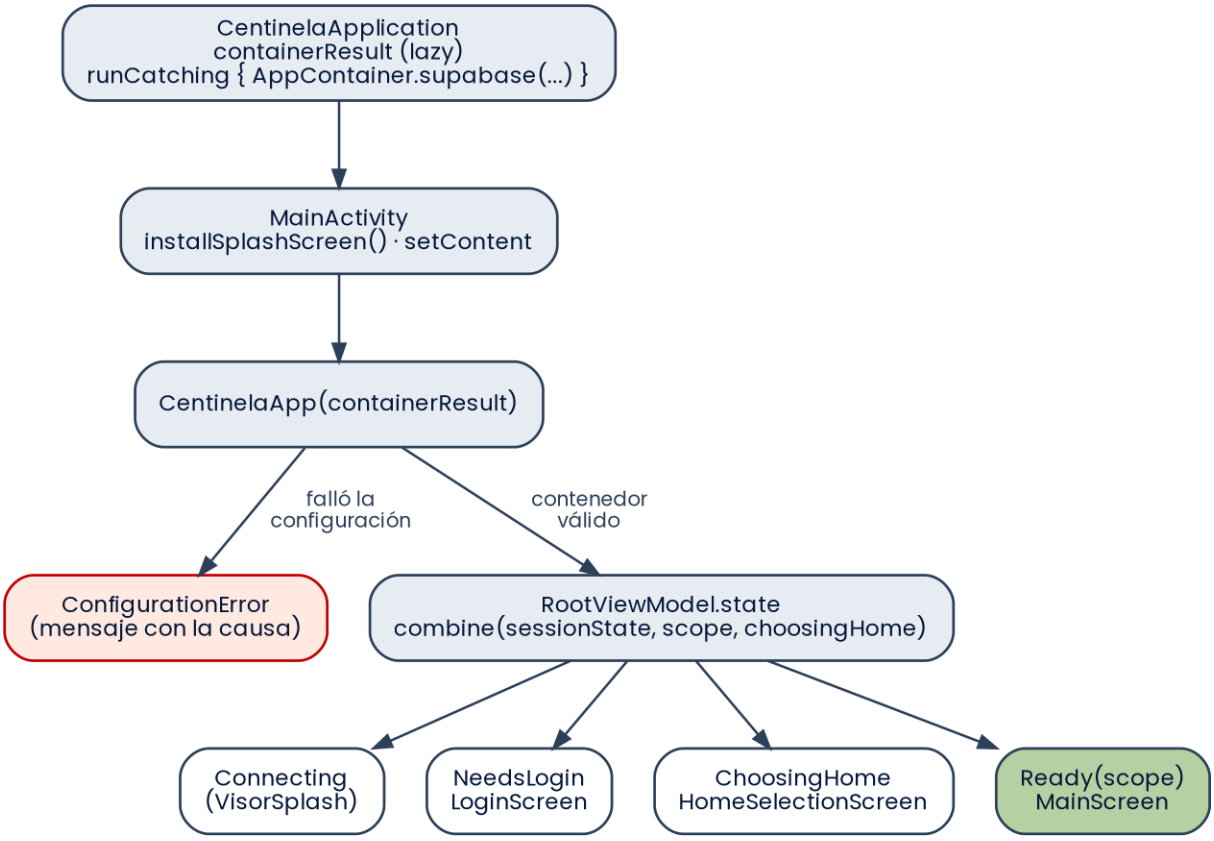


Figura 3. Flujo de arranque y estados raíz (RootState).

Listado 7 · Contenedor diferido con captura del error de configuración
`app/src/main/kotlin/com/robotcentinela/app/shell/CentinelaApplication.kt:13`

```
class CentinelaApplication : Application() {

    /** Contenedor, o el error de configuración que impide crearlo. */
    val containerResult: Result<AppContainer> by lazy {
        runCatching { AppContainer.supabase(SupabaseConfig.fromBuildConfig(), publicRead =
            BuildConfig.DEMO_PUBLIC_READ) }
    }
}
```

Listado 8 · RootViewModel: de la sesión y el domicilio a la pantalla
`app/src/main/kotlin/com/robotcentinela/app/ui/app/RootViewModel.kt:45`

```
val state: StateFlow<RootState> = combine(
    container.authPort.sessionState,
    container.currentHomeScope.scope,
    choosingHome,
) { session, scope, choosing ->
    when (session) {
        SessionState.Initializing -> RootState.Connecting
        is SessionState.NotAuthenticated -> RootState.NeedsLogin
        // Con la renovación fallida (p. ej. sin red) se sigue mostrando la app: cada pantalla informa
        // su error y el SDK reintenta la renovación.
        is SessionState.Authenticated, SessionState.RefreshFailed ->
            if (scope == null || choosing) RootState.ChoosingHome else RootState.Ready(scope)
    }
}.stateIn(viewModelScope, SharingStarted.Eagerly, RootState.Connecting)
```

Detalle de sesión

Con `SessionState.RefreshFailed` (por ejemplo, sin red) la app **sigue mostrando contenido**: cada pantalla informa su propio error y el SDK reintenta la renovación del token. Solo `NotAuthenticated` regresa al inicio de sesión.

4. Estructura de archivos

4.1 Árbol del repositorio

Listado 9 · Estructura del repositorio (resumida)

```
Robot_Centinela-Aplicacion_Movil/
├── settings.gradle.kts            módulos :core y :app; repositorios google() y mavenCentral()
├── build.gradle.kts · gradle.properties · gradlew
├── gradle/libs.versions.toml      catálogo de versiones (única fuente de versiones)
├── local.properties(.example)     configuración local, NO se versiona
├── core/                          Kotlin/JVM puro (44 archivos main, 10 clases de prueba)
│   └── src/main/kotlin/com/robotcentinela/app/
│       ├── identity/ telemetry/ environmentalmonitoring/ robothealth/
│       ├── incidents/ missions/ homemapping/     (cada uno: domain · ports · application)
│       └── shared/                errores, paginación, intervalos UTC
├── app/                           Android application (59 archivos Kotlin main)
│   ├── build.gradle.kts · proguard-rules.pro · OFL-Poppins.txt · OFL-Questrial.txt
│   └── src/
│       ├── main/
│       │   ├── AndroidManifest.xml
│       │   ├── kotlin/com/robotcentinela/app/
│       │   │   ├── <dominio>/adapters/supabase/   adaptadores y DTO
│       │   │   ├── shell/                         AppContainer, Adapters, supabase/
│       │   │   └── ui/                            app · auth · homes · main · summary · map ·
│       │   │                                      missions · history · events · settings ·
│       │   │                                      components · model · theme
│       │   └── res/               drawable (íconos y marca) · font · mipmap-anydpi-v26 ·
│       │                          values · values-night
│       └── test/
│           ├── kotlin/…           pruebas de adaptadores, configuración y presentación
│           └── resources/fixtures/telemetry/   frames JSON sintéticos
├── docs/                          architecture · backend · data_dictionary · decisions (ADR) ·
│                                  releases · requirements · ui · manual-de-usuario · manual-tecnico
└── tools/identidad/               scripts que vectorizan el isotipo y los íconos de la marca
```

4.2 Convenciones

- **Paquete raíz único** `com.robotcentinela.app`, igual en `:core` y `:app` (el `applicationId` histórico no cambió con la marca V.I.S.O.R.).
- **Por dominio, no por capa técnica:** `telemetry/…` agrupa su dominio, puertos, casos de uso y su adaptador; la primera lectura del árbol revela el negocio.
- **Sufijos:** `…Port` (interfaz de salida), `…Adapter` (implementación), `…Dto` (forma de la fila remota), `…ViewModel`, `…Screen`/`…Sheet` (Compose), `…Presentation` (modelos de UI sin Compose).
- **Nombres de recursos de Supabase** únicamente en `shell/supabase/SupabaseResources.kt`; cambiar una tabla o columna se limita a ese archivo y al DTO afectado.
- **Tests espejo:** cada clase de `:core` tiene su prueba en la misma ruta bajo `src/test`; los datos de prueba son sintéticos y viven en `fixtures/`.
- **Comentarios en español** y KDoc en las clases públicas; las decisiones no obvias remiten a un ADR.

4.3 Dónde hacer cada tipo de cambio

Quiero…	Se toca
Cambiar una regla de interpretación (calidad, frescura, batería)	`:core` · `…/domain` y su prueba; ningún cambio en `:app`
Cambiar el nombre de una tabla o columna del backend	`SupabaseResources.kt`, el DTO del adaptador y, si es el frame, `NormalizedFrameAssembler`
Soportar otro origen de datos (REST propio, MQTT directo)	Un adaptador nuevo que implemente el puerto y su alta en `Adapters.kt`
Agregar una pantalla	`ui/<pantalla>/` (Screen + ViewModel), ruta en `MainScreen.kt`, dependencias vía `uiDependencies`
Ajustar color, tipografía o formas	`ui/theme/` (`CentinelaColors`, `Type`, `Theme`); no hay hexadecimales sueltos en las pantallas
Ajustar un parámetro de sincronización o frescura	Constructor de `SyncConfig` / `FreshnessPolicy` / `IngestConnectivityPolicy` en `AppContainer` (ADR-004)

5. Conexión con el backend

5.1 Configuración y validación

La URL y la clave pública se leen de `local.properties` durante la compilación y se incrustan en `BuildConfig` (`SUPABASE_URL`, `SUPABASE_PUBLISHABLE_KEY`). `SupabaseConfig` valida los valores **al crear el contenedor**: exige URL HTTPS y rechaza claves privilegiadas, tanto las nuevas (`sb_secret_…`) como los JWT con `role = service_role`, decodificando su carga útil.

Listado 10 · SupabaseConfig.create `app/src/main/kotlin/com/robotcentinela/app/shell/supabase/SupabaseConfig.kt:37`

```
fun create(url: String, publishableKey: String): SupabaseConfig {
    val cleanUrl = url.trim()
    val cleanKey = publishableKey.trim()
    check(cleanUrl.isNotEmpty()) { "Falta supabase.url en local.properties" }
    check(cleanKey.isNotEmpty()) { "Falta supabase.publishableKey en local.properties" }
    check(cleanUrl.startsWith("https://")) { "supabase.url debe usar HTTPS" }
    check(!isPrivilegedKey(cleanKey)) {
        "supabase.publishableKey es una clave privilegiada; la app solo admite la clave pública"
    }
    return SupabaseConfig(cleanUrl, cleanKey)
}
```

Listado 11 · Detección de claves privilegiadas
`app/src/main/kotlin/com/robotcentinela/app/shell/supabase/SupabaseConfig.kt:50`

```
internal fun isPrivilegedKey(key: String): Boolean {
    if (key.startsWith("sb_secret_")) return true
    val parts = key.split('.')
    if (parts.size != 3) return false
    val role = runCatching {
        val payload = String(Base64.getUrlDecoder().decode(parts[1]), Charsets.UTF_8)
        Json.parseToJsonElement(payload).jsonObject["role"]?.jsonPrimitive?.content
    }.getOrNull()
    return role == "service_role"
}
```

`toString()` omite la clave y los mensajes de error nunca incluyen payloads ni cuerpos de respuesta, de modo que ni los registros ni las pantallas de error filtran secretos.

5.2 Cliente de Supabase

`SupabaseClientFactory` crea un único cliente con tres módulos. **Storage no se instala**: el contrato de mapas está pendiente. El `Json` compartido usa `ignoreUnknownKeys` para que los cambios aditivos del contrato v1 no rompan la app.

Listado 12 · Cliente con Auth, Postgrest y Realtime
`app/src/main/kotlin/com/robotcentinela/app/shell/supabase/SupabaseClientFactory.kt:22`

```
val json = Json { ignoreUnknownKeys = true }

fun create(config: SupabaseConfig): SupabaseClient =
    createSupabaseClient(supabaseUrl = config.url, supabaseKey = config.publishableKey) {
        defaultSerializer = KotlinXSerializer(json)
        install(Auth)
        install(Postgrest)
        install(Realtime)
    }
```

5.3 Autenticación y sesión

La sesión (tokens, renovación y persistencia) la gestiona el SDK. `SupabaseAuthAdapter` solo traduce `SessionStatus` al dominio (`SessionState`) y expone `signInWithEmail`, `signOut` y `requestAccessRecovery`. El método es **correo y contraseña**, porque la recuperación de acceso forma parte del flujo (ADR-003, ADR-006).

Listado 13 · Sesión observable e inicio de sesión
`app/src/main/kotlin/com/robotcentinela/app/identity/adapters/supabase/SupabaseAuthAdapter.kt:29`

```
override val sessionState: StateFlow<SessionState> = supabase.auth.sessionStatus
    .map(::toDomain)
    .stateIn(scope, SharingStarted.Eagerly, SessionState.Initializing)

override suspend fun signInWithEmail(email: String, password: String) = supabaseCall("inicio de
    sesión") {
    supabase.auth.signInWith(Email) {
        this.email = email
        this.password = password
    }
}
```

Listado 14 · Traducción de estados del SDK al dominio
`app/src/main/kotlin/com/robotcentinela/app/identity/adapters/supabase/SupabaseAuthAdapter.kt:48`

```
private fun toDomain(status: SessionStatus): SessionState = when (status) {
    is SessionStatus.Authenticated -> status.session.user?.id
        ?.let { SessionState.Authenticated(UserId(it)) }
        ?: SessionState.NotAuthenticated(signedOut = false)
    is SessionStatus.NotAuthenticated -> SessionState.NotAuthenticated(signedOut = status.isSignOut)
    is SessionStatus.RefreshFailure -> SessionState.RefreshFailed
    SessionStatus.Initializing -> SessionState.Initializing
}
```

SessionStatus (SDK)	SessionState (dominio)	Efecto en RootState
`Initializing`	`Initializing`	`Connecting` (splash de la marca)
`Authenticated`	`Authenticated(UserId)`	`ChoosingHome` o `Ready` según el ámbito
`NotAuthenticated(isSignOut)`	`NotAuthenticated(signedOut)`	`NeedsLogin`
`RefreshFailure`	`RefreshFailed`	Se mantiene la app; cada pantalla informa su error

5.4 Ámbito de domicilio y autorización

El **servidor** autoriza mediante RLS; el cliente no decide permisos. Tras iniciar sesión, `SupabaseHomeScopeAdapter` lee las membresías del usuario con el domicilio embebido, y `SelectHomeScope` fija el **ámbito** (`HomeScope`: domicilio y robots) en `CurrentHomeScope`. Cada caso de uso que toca un robot llama a `requireRobot`, que lanza `NotInScopeException` si el robot no pertenece al domicilio elegido: es una defensa del cliente que evita consultas fuera de ámbito, **no** una medida de seguridad.

Listado 15 · Membresías con el domicilio embebido (la sesión del SDK aporta user_id)
`app/src/main/kotlin/com/robotcentinela/app/identity/adapters/supabase/SupabaseHomeScopeAdapter.kt:34`

```
override suspend fun memberships(): List<HomeMembership> = supabaseCall("membresías") {
    val userId = supabase.auth.currentUserOrNull()?.id
        ?: throw AccessDeniedException("No hay una sesión válida")
    supabase.from(SupabaseResources.MEMBERSHIPS)
        .select(PostgrestColumns.raw("role, status, ${SupabaseResources.HOMES}(id, name, timezone)")) {
            filter { eq(Columns.USER_ID, userId) }
        }
        .decodeList<MembershipRowDto>()
        .map { it.toDomain() }
}
```

Listado 16 · Comprobación de ámbito en el cliente
`core/src/main/kotlin/com/robotcentinela/app/identity/application/CurrentHomeScope.kt:37`

```
fun requireRobot(robotId: RobotId): HomeScope {
    val scope = current.value ?: throw NotInScopeException("No hay un domicilio seleccionado")
    if (!scope.contains(robotId)) {
        throw NotInScopeException("El robot no pertenece al domicilio seleccionado")
    }
    return scope
}
```

> **Roles**
> La app guarda el texto de `memberships.role` (`reader`/`technician`) pero **no autoriza nada por rol**. Todos los permisos de lectura los resuelven las políticas RLS (`docs/backend/propuesta-politicas-lectura.sql`: lectura por membresía `active`).

5.5 Recursos consultados

Operación	Recurso y consulta	Filtros y orden
Membresías	`memberships` con `homes(id,name,timezone)` embebido	`user_id = sesión`
Robots	`robots`: `id, home_id, public_code, status`	`home_id`
Snapshot	`telemetry_frames` + hijas (ver 6.3)	`robot_id`, `stream_version DESC`, límite 1
Deltas	`telemetry_frames` + hijas	`robot_id`, `stream_version > cursor`, ASC, límite = página (100)
Historial	`telemetry_frames` + hijas	`robot_id`, `captured_at` en [inicio, fin), `mission_id` opcional, `range(offset, offset+limit-1)`

Operación	Recurso y consulta	Filtros y orden
Alertas	`alerts` (9 columnas)	`robot_id`, intervalo de `opened_at`, tipo/severidad/estado, `opened_at DESC`, paginado
Capacidades	`channels`: `channel_key, metric_type, unit_code`	`robot_id`
Misiones	`missions`: 7 columnas	`robot_id`, orden por `started_at`
Tiempo real	Realtime · Postgres Changes · INSERT en `telemetry_frames`	filtro `robot_id = <uuid>`

Listado 17 · Consulta de deltas por cursor `app/src/main/kotlin/com/robotcentinela/app/telemetry/adapters/supabase/SupabaseTelemetryQueryAdapter.kt:52`

```
override suspend fun framesAfter(
    robotId: RobotId,
    after: StreamVersion,
    limit: Int,
): List<FrameRecord> = supabaseCall("deltas de telemetría") {
    supabase.from(SupabaseResources.TELEMETRY_FRAMES_TABLE).select(frameColumns) {
        filter {
            eq(Columns.ROBOT_ID, robotId.value.toString())
            gt(Columns.STREAM_VERSION, after.value)
        }
        order(Columns.STREAM_VERSION, Order.ASCENDING)
        limit(limit.toLong())
    }.decodeList<JsonObject>().map(::toRecord)
}
```

5.6 Traducción de errores

Los adaptadores envuelven cada llamada en `supabaseCall`, que convierte las excepciones del SDK en cuatro tipos propios de `:core` (`CentinelaException`). Así ni el dominio ni la UI conocen Ktor o supabase-kt, y un fallo de red nunca se interpreta como robot apagado.

Excepción	Cuándo se produce	Qué hace la UI
`AccessDeniedException`	HTTP 401/403 o sin sesión	Mensaje de acceso; el login se muestra si la sesión cayó
`NotInScopeException`	El robot no pertenece al domicilio elegido	Estado de error del sincronizador
`DataAccessException`	Otro código HTTP o error de red	Mensaje con reintento; datos previos se conservan
`ContractViolationException`	Respuesta no deserializable o valor inválido	Frame rechazado o error de consulta

Listado 18 · supabaseCall `app/src/main/kotlin/com/robotcentinela/app/shell/supabase/SupabaseErrors.kt:19`

```
internal suspend inline fun <T> supabaseCall(operation: String, block: () -> T): T =
```

5.7 Transporte y límites

- **HTTPS obligatorio**; Realtime viaja por WebSocket seguro bajo el mismo host. No hay certificate pinning.
- **Sin reintentos propios** en las consultas: la resiliencia está en el sincronizador (reconciliación a los 30 s y en cada aviso) y en los sondeos de las pantallas.
- **Paginación explícita** (`PageRequest`): 500 filas en el historial crudo y en las alertas del historial, 20 por página en Eventos, 5 y 20 en el resumen. No se descargan tablas completas.
- **Permiso** `INTERNET` es el único del manifiesto; `allowBackup=false`.

6. Recepción de datos y actualizaciones en tiempo real

Esta es la parte más delicada de la app. El diseño parte de una regla: **el aviso en tiempo real nunca es el dato**. Realtime solo indica «hay algo nuevo confirmado»; los datos se obtienen siempre con una consulta ordenada por `stream_version`, el cursor de commit por robot. Esto hace la sincronización tolerante a avisos perdidos, repetidos o desordenados.

6.1 Flujo de extremo a extremo

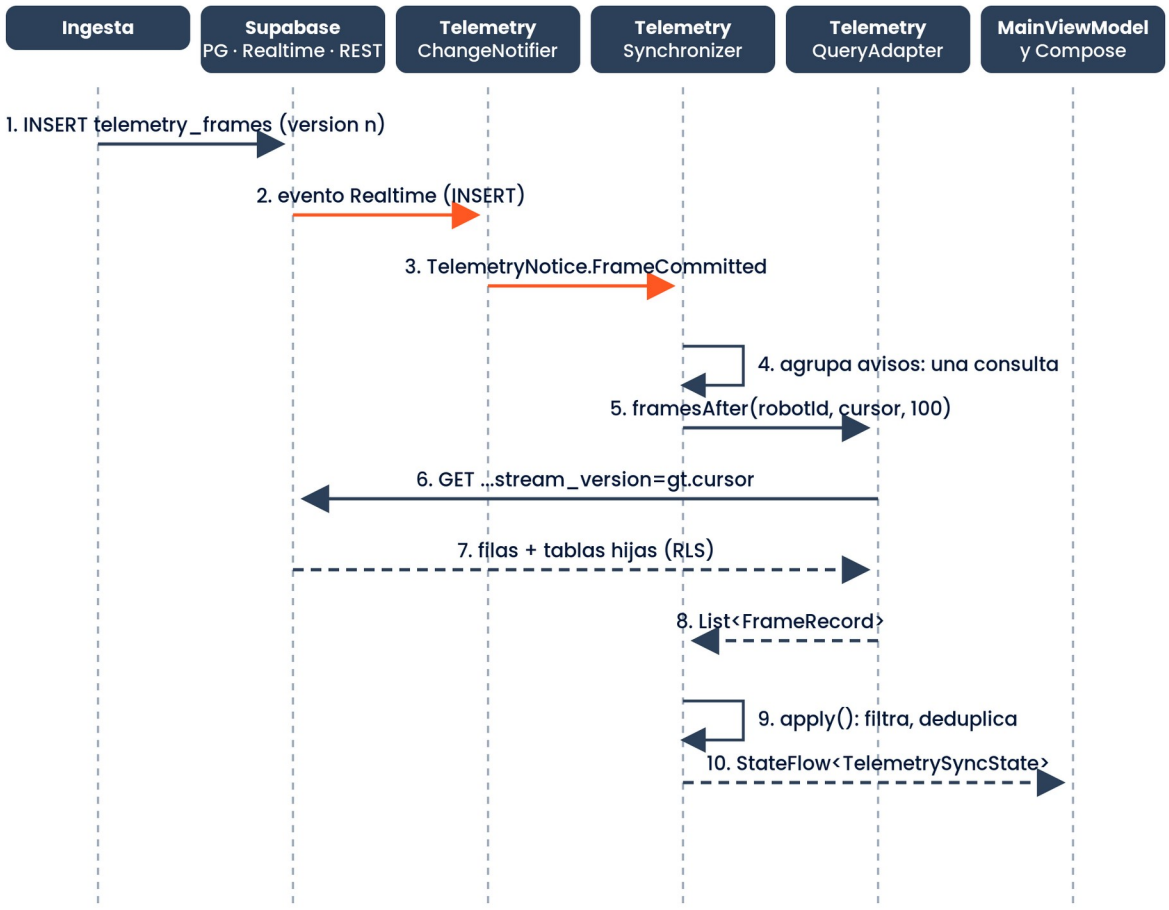


Figura 4. Secuencia de una actualización. Los pasos 2 y 3 (naranja) son solo avisos; los datos llegan en los pasos 5 a 8.

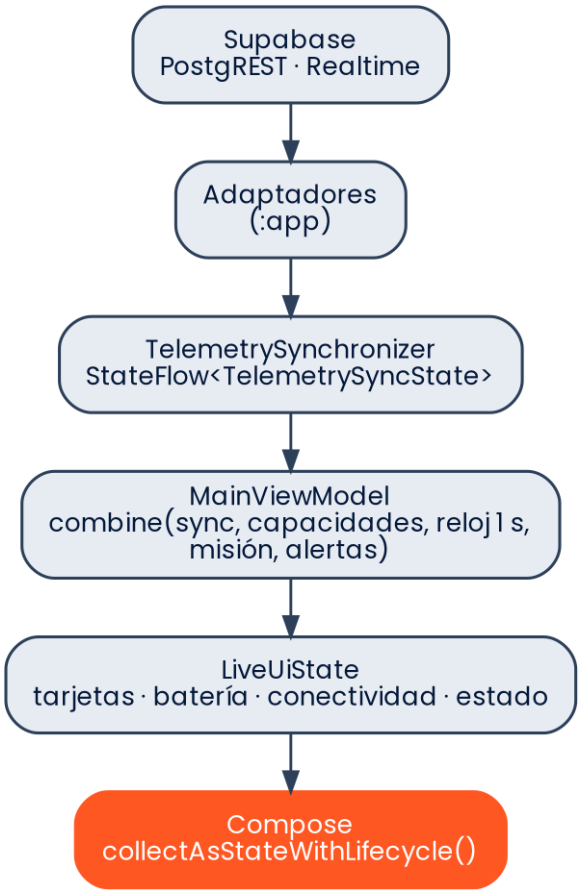


Figura 5. Del backend a la pantalla.

6.2 Aviso en tiempo real: SupabaseTelemetryChangeNotifier

El adaptador abre un canal por robot (`telemetry-<robotId>`), escucha **Postgres Changes** de tipo INSERT sobre `telemetry_frames` con filtro `robot_id`, y traduce el estado del canal a `TelemetryNotice`. Al cancelar el flujo, retira el canal dentro de `NonCancellable` para no filtrar suscripciones. Requiere que `telemetry_frames` esté en la publicación `supabase_realtime` (B4).

Listado 19 · Notificador de cambios (Realtime)
`app/src/main/kotlin/com/robotcentinela/app/telemetry/adapters/supabase/SupabaseTelemetryChangeNotifier.kt:31`

```
override fun notices(robotId: RobotId): Flow<TelemetryNotice> = channelFlow {
    val channel = supabase.channel("telemetry-${robotId.value}")
    val inserts = channel.postgresChangeFlow<PostgresAction.Insert>(schema = SupabaseResources.SCHEMA) {
        table = SupabaseResources.TELEMETRY_FRAMES_TABLE
        filter(SupabaseResources.Columns.ROBOT_ID, FilterOperator.EQ, robotId.value.toString())
    }

    launch { inserts.collect { send(TelemetryNotice.FrameCommitted) } }
    launch {
        var wasSubscribed = false
        channel.status.collect { status ->
```

```
            when (status) {
                RealtimeChannel.Status.SUBSCRIBED -> {
                    wasSubscribed = true
                    send(TelemetryNotice.Subscribed)
                }
                RealtimeChannel.Status.UNSUBSCRIBED -> if (wasSubscribed) {
                    wasSubscribed = false
                    send(TelemetryNotice.Interrupted("Canal Realtime desconectado"))
                }
                else -> Unit
            }
        }
    }

    try {
        channel.subscribe()
        awaitCancellation()
    } finally {
        withContext(NonCancellable) { supabase.realtime.removeChannel(channel) }
    }
}
```

TelemetryNotice	Significado	Efecto en el sincronizador
`Subscribed`	El canal quedó suscrito	Marca «suscrito» y fuerza una sincronización (cubre lo ocurrido antes de suscribirse)
`FrameCommitted`	Se confirmó una inserción	Si hay suscripción, programa una sincronización
`Interrupted(reason)`	Canal desconectado o flujo terminado	Estado `Offline(reason)`; se espera la nueva suscripción

6.3 Consulta de datos y reconstrucción del frame

El frame no se guarda como JSON: se reparte en tablas. `NormalizedFrameAssembler.SELECT` pide en **una sola consulta** PostgREST la cabecera con sus relaciones embebidas, y `toContractRow` vuelve a armar el JSON del contrato v1.1 para que un único mapeador (`TelemetryFrameMapper`) aplique las reglas de siempre. Si el esquema cambia, solo se ajusta el ensamblador.

Listado 20 · Consulta con relaciones embebidas (sintaxis PostgREST)
`app/src/main/kotlin/com/robotcentinela/app/telemetry/adapters/supabase/NormalizedFrameAssembler.kt:39`

```
const val SELECT =
    "id, robot_id, mission_id, sequence, captured_at, received_at, monotonic_ms, clock_quality,
        stream_version, " +
        "device_sessions(boot_id, devices(id, firmware_version)), " +
        "sensor_readings(value, boolean_value, quality, calibration_id, channels(channel_key)), " +
        "frame_poses(map_version_id, x_m, y_m, theta_rad, sigma_position_m, quality), " +
        "robot_states(reported_state, motors_active), " +
        "battery_readings(voltage_v, current_a, soc_pct, method_code, quality)"
```

Listado 21 · Reconstrucción del contrato: sesión de arranque y dispositivo
`app/src/main/kotlin/com/robotcentinela/app/telemetry/adapters/supabase/NormalizedFrameAssembler.kt:73`

```
val session = row.one("device_sessions")
val device = session?.one("devices")
(session?.get("boot_id").orNull() ?: payload?.get("boot_id").orNull())?.let { put("boot_id", it) }
(device?.get("id").orNull() ?: payload?.get("device_id").orNull())?.let { put("device_id", it) }
(device?.get("firmware_version").orNull() ?: payload?.get("firmware_version").orNull())
    ?.let { put("firmware_version", it) }
```

Listado 22 · Mapeador: un frame inválido se rechaza, nunca detiene el cursor
`app/src/main/kotlin/com/robotcentinela/app/telemetry/adapters/supabase/TelemetryFrameMapper.kt:56`

```
fun fromRow(row: JsonObject): FrameRecord {
    val version = row["stream_version"]?.jsonPrimitive?.longOrNull
        ?.takeIf { it >= 0 }
        ?: throw ContractViolationException("La fila de frame no trae un stream_version válido")
    val streamVersion = StreamVersion(version)
    val frameId = row["event_id"]?.jsonPrimitive?.contentOrNull
        ?.let { runCatching { FrameId(UUID.fromString(it)) }.getOrNull() }

    return try {
        val dto = SupabaseClientFactory.json.decodeFromJsonElement(TelemetryFrameRowDto.serializer(),
            row)
        FrameRecord.Interpreted(toDomain(dto, streamVersion))
    } catch (e: SerializationException) {
        FrameRecord.Rejected(streamVersion, frameId, "El frame no cumple la estructura del contrato v1")
    } catch (e: FrameContractException) {
        FrameRecord.Rejected(streamVersion, frameId, e.message.orEmpty())
    } catch (e: IllegalArgumentException) {
        // Invariantes del dominio (por ejemplo, pose no finita o channel_key repetido).
        FrameRecord.Rejected(streamVersion, frameId, e.message ?: "El frame viola una regla del
            dominio")
    }
}
```

`FrameRecord.Rejected` conserva `stream_version` y el motivo (sin payload). El sincronizador **avanza el cursor y cuenta** los rechazados (`rejectedFrames`), de modo que un frame defectuoso no bloquea el flujo ni desaparece sin dejar rastro.

6.4 El sincronizador paso a paso

`TelemetrySynchronizer` (en `:core`, sin dependencias de Android) recibe dos puertos —consulta y notificador— y publica un `StateFlow<TelemetrySyncState>` con la proyección del último dato, el cursor y el estado de conexión. Su bucle principal:

1. **Aviso → buffer acotado.** Los avisos entran a un `Channel` de capacidad 64 con `DROP_OLDEST`. Perder un aviso viejo es inocuo: cualquier aviso posterior dispara la misma consulta por cursor, y la memoria no crece sin límite.
2. **Espera con tiempo máximo.** El bucle espera un aviso hasta `reconciliationInterval` (30 s). Si vence sin avisos y hay suscripción, **reconcilia** de todos modos (cubre avisos omitidos).
3. **Agrupación.** Tras el primer aviso se drenan los pendientes; varios avisos seguidos producen **una sola** consulta.
4. **Primera vez: snapshot.** Sin cursor se pide el frame más reciente y se fija el cursor en su `stream_version` (marca de agua).
5. **Deltas.** `framesAfter(cursor, 100)` por páginas hasta que una página llega incompleta o el cursor no avanza.
6. **Hueco de retención.** Si el primer frame recibido no es `cursor + 1`, los intermedios ya no existen: se toma un snapshot nuevo y se marca `historyLimited`.

7. **Aplicación.** Los registros se ordenan, se descartan los ≤ cursor, se deduplica por `frame_id` (ventana de 2 700) y se actualiza la proyección; los rechazados solo suman al contador.

Listado 23 · Bucle principal: avisos, agrupación y reconciliación
`core/src/main/kotlin/com/robotcentinela/app/telemetry/application/TelemetrySynchronizer.kt:78`

```
private suspend fun run(): Unit = coroutineScope {
    setStatus(if (state.value.lastAppliedVersion == null) SyncStatus.Connecting else
        SyncStatus.Reconciling)

    // Buffer acotado con descarte del aviso más antiguo para evitar crecimiento ilimitado de memoria.
    val notices = Channel<TelemetryNotice>(capacity = 64, onBufferOverflow = BufferOverflow.DROP_OLDEST)
    launch {
        while (true) {
            notifier.notices(robotId)
                .catch { cause -> emit(TelemetryNotice.Interrupted(cause.message ?: "Suscripción
                    interrumpida")) }
                .collect { notices.send(it) }
            // Si el flujo de avisos termina, se reintenta la suscripción tras el intervalo.
            notices.send(TelemetryNotice.Interrupted("Suscripción finalizada"))
            delay(config.reconciliationInterval)
        }
    }

    var subscribed = false
    while (true) {
        // Sin avisos durante el intervalo: reconciliación periódica (§7.4).
        val first = withTimeoutOrNull(config.reconciliationInterval) { notices.receive() }
        var mustSync = first == null && subscribed

        // Se agrupan los avisos pendientes: varios avisos seguidos producen una sola consulta.
        var notice: TelemetryNotice? = first
        while (notice != null) {
            when (notice) {
                TelemetryNotice.Subscribed -> {
                    subscribed = true
                    mustSync = true
                }
                TelemetryNotice.FrameCommitted -> if (subscribed) mustSync = true
                is TelemetryNotice.Interrupted -> {
                    subscribed = false
                    mustSync = false
                    setStatus(SyncStatus.Offline(notice.reason))
                }
            }
            notice = notices.tryReceive().getOrNull()
        }

        if (mustSync) synchronize()
    }
}
```

Listado 24 · Deltas por cursor y detección de hueco
`core/src/main/kotlin/com/robotcentinela/app/telemetry/application/TelemetrySynchronizer.kt:141`

```
private suspend fun catchUp() {
    var snapshotTaken = false
    while (true) {
        val after = state.value.lastAppliedVersion ?: StreamVersion.NONE
        val page = query.framesAfter(robotId, after, config.deltaPageSize).sortedBy { it.streamVersion }
        val first = page.firstOrNull() ?: return

        if (first.streamVersion > after.next() && !snapshotTaken) {
            // Hueco en el cursor: los frames intermedios ya no existen (retención).
```

```
            applySnapshot(historyLimited = true)
            snapshotTaken = true
            continue
        }

        apply(page)
        val advanced = state.value.lastAppliedVersion != after
        if (page.size < config.deltaPageSize || !advanced) return
    }
}
```

Listado 25 · Aplicación ordenada, sin retroceso y sin duplicados
`core/src/main/kotlin/com/robotcentinela/app/telemetry/application/TelemetrySynchronizer.kt:176`

```
private suspend fun apply(records: List<FrameRecord>) {
    var current = state.value
    val applied = mutableListOf<TelemetryFrame>()

    for (record in records.sortedBy { it.streamVersion }) {
        val last = current.lastAppliedVersion
        if (last != null && record.streamVersion <= last) continue

        current = when (record) {
            is FrameRecord.Interpreted -> if (recentFrameIds.add(record.frame.id)) {
                applied += record.frame
                current.copy(projection = current.projection.apply(record.frame))
            } else {
                current
            }
            is FrameRecord.Rejected -> current.copy(rejectedFrames = current.rejectedFrames + 1)
        }
        current = current.copy(lastAppliedVersion = record.streamVersion)
    }

    mutableState.value = current
    applied.forEach { newFrames.emit(it) }
}
```

6.5 Estados de sincronización

`SyncStatus` describe la condición **real**; nunca se muestra «en vivo» sin estarlo (por ejemplo, tras `stop()` queda `Stopped`).

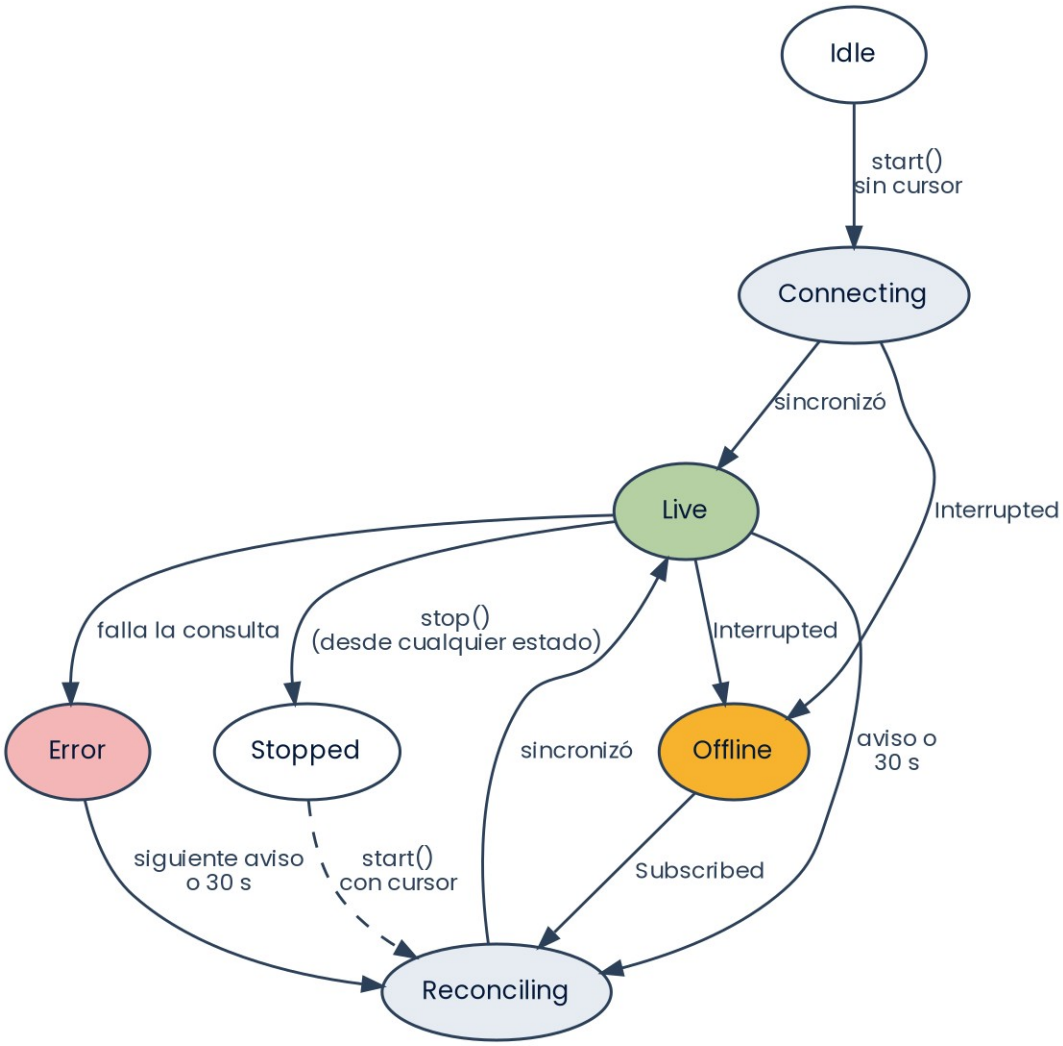


Figura 6. Máquina de estados de SyncStatus (líneas punteadas: transiciones al iniciar).

Estado	Etiqueta en la app	Cuándo
`Idle`	Inactiva	Antes de `start()`
`Connecting`	Conectando	Primer arranque, sin cursor
`Reconciling`	Actualizando	Recuperando deltas tras reconexión o regreso desde segundo plano
`Live`	En vivo	Suscrito y al día con los frames confirmados
`Offline(reason)`	Sin conexión	La suscripción se interrumpió; se muestra la última captura con su hora
`Error(message)`	Error de consulta	Falló una consulta; se reintenta con el siguiente aviso o a los 30 s
`Stopped`	En pausa	App en segundo plano o sincronizador detenido

6.6 Parámetros

Listado 26 · SyncConfig `core/src/main/kotlin/com/robotcentinela/app/telemetry/application/TelemetrySyncState.kt:72`

```
data class SyncConfig(
    val deltaPageSize: Int = 100,
    val reconciliationInterval: Duration = 30.seconds,
    val dedupeWindow: Int = 2_700,
)
```

Parámetro	Valor	Origen y criterio
`deltaPageSize`	100	Provisional; ajustar con pruebas de carga
`reconciliationInterval`	30 s	Valor inicial del expediente (§7.4); cubre avisos omitidos
`dedupeWindow`	2 700 `frame_id`	Una misión de 90 min a un frame cada 2 s
Buffer de avisos	64, `DROP_OLDEST`	Interno de `run()`; acota memoria sin perder datos
`appliedFrames`	`SharedFlow`, buffer extra 64	Frames nuevos en orden; ninguna pantalla lo consume hoy

Todos los valores son **iniciales** (ADR-004): viven como parámetros por defecto en sus constructores y se ajustan en `AppContainer` sin tocar la lógica.

6.7 Ciclo de vida y llegada a la interfaz

La sincronización solo corre mientras `MainScreen` está visible: `LifecycleStartEffect` llama a `MainViewModel.onStart()` y `onStop()`. Al volver del segundo plano, `start()` reconcilia **desde el último cursor aplicado**, sin repetir el snapshot. El ViewModel combina el estado del sincronizador con un reloj de 1 s —que recalcula la antigüedad de cada canal sin consultar la red—, las capacidades y las alertas, y publica un único `LiveUiState` con `stateIn(WhileSubscribed(5 s))`.

Listado 27 · Sesión en vivo: sincronizador, misión (10 s) y alertas (30 s)
`app/src/main/kotlin/com/robotcentinela/app/ui/main/MainViewModel.kt:182`

```
private fun startSession(robot: Robot) {
    val sync = synchronizer ?: runCatching { container.telemetrySynchronizer(robot.id)
```

```
        }.getOrNull()?.also { synchronizer = it }
    if (sync == null) {
        syncState.value = TelemetrySyncState(status = SyncStatus.Error("El robot no pertenece al
            domicilio seleccionado"))
        return
    }
    val jobs = mutableListOf<Job>()
    // Se suscribe al estado antes de arrancar para no perder el snapshot inicial.
    jobs += viewModelScope.launch(start = CoroutineStart.UNDISPATCHED) {
        sync.state.collect { syncState.value = it }
    }
    jobs += viewModelScope.launch {
        syncState.map { it.projection.latestFrame?.missionId }.distinctUntilChanged().collectLatest {
            missionId ->
            while (true) {
                refreshMission(robot, missionId)
                delay(10_000)
            }
        }
    }
    jobs += viewModelScope.launch {
        alertRange.collectLatest {
            while (true) {
                refreshAlerts(robot)
                delay(30_000)
            }
        }
    }
    if (capabilities.value.isEmpty()) {
        jobs += viewModelScope.launch {
            container.queryChannelCapabilities?.let { query ->
                capabilities.value = runCatching { query(robot.id) }.getOrDefault(emptyList())
            }
        }
    }
    sessionJobs = jobs
    sync.start(viewModelScope)
}
```

Origen	Frecuencia	Mecanismo
Telemetría	Por aviso + reconciliación a los 30 s	Realtime + deltas por `stream_version`
Edad de los datos	Cada 1 s	Reloj interno del ViewModel; sin red
Alertas	Cada 30 s (y al cambiar el rango)	Sondeo con `queryAlertHistory`
Misión actual	Cada 10 s	Sondeo con `getMissionSummary`
Capacidades de canal	Una vez por sesión	Consulta de `channels`
Historial	A demanda	`framesBetween` paginado (500 filas)

6.8 Historial por intervalo

`QueryTelemetryHistory` convierte fechas **locales** del domicilio a un intervalo UTC semiabierto con `UtcInterval.fromLocalDates` y la zona del domicilio (por defecto `America/Mexico_City`). Devuelve los frames tal como se persistieron: sin cambiar unidades y sin rellenar huecos.

Listado 28 · Historial por fechas locales `core/src/main/kotlin/com/robotcentinela/app/telemetry/application/QueryTelemetryHistory.kt:27`

```
suspend operator fun invoke(
    robotId: RobotId,
    from: LocalDate,
    toInclusive: LocalDate,
    page: PageRequest,
    missionId: MissionId? = null,
): List<FrameRecord> {
    val scope = currentScope.requireRobot(robotId)
    val interval = UtcInterval.fromLocalDates(from, toInclusive, scope.home.timezone)
    return query.framesBetween(robotId, interval, missionId, page)
}
```

7. Modelo de datos y diccionario de datos

Hay tres representaciones del mismo dato y conviene no confundirlas: el **contrato del frame v1.1** (el JSON que emite el robot), el **esquema normalizado de Supabase** (cómo se guarda) y el **modelo de dominio** de `:core` (cómo lo usa la app). Este capítulo las documenta y las relaciona.

7.1 Contrato del frame v1.1

Un frame es una captura completa de todos los sensores en un instante. El DTO `TelemetryFrameRowDto` replica la estructura del contrato y agrega `received_at` y `stream_version`, que asigna el backend al persistir. Un frame con otra **versión mayor** se rechaza.

Listado 29 · DTO del frame (TelemetryFrameRowDto)
`app/src/main/kotlin/com/robotcentinela/app/telemetry/adapters/supabase/TelemetryFrameDto.kt:14`

```
data class TelemetryFrameRowDto(
    @SerialName("schema_version") val schemaVersion: String,
    @SerialName("event_id") val eventId: String,
    @SerialName("device_id") val deviceId: String,
    @SerialName("boot_id") val bootId: String,
    val sequence: Long,
    @SerialName("captured_at") val capturedAt: String? = null,
    @SerialName("monotonic_ms") val monotonicMs: Long,
    @SerialName("clock_quality") val clockQuality: String,
    @SerialName("mission_id") val missionId: String? = null,
    @SerialName("map_version_id") val mapVersionId: String? = null,
    val pose: PoseDto? = null,
    val readings: List<ReadingDto>,
    val battery: BatteryDto? = null,
    @SerialName("robot_state") val robotState: String? = null,
    @SerialName("motors_active") val motorsActive: Boolean? = null,
    @SerialName("firmware_version") val firmwareVersion: String? = null,
    // Campos asignados por el backend (§7.2: "El backend asigna received_at y stream_version").
    // PROVISIONAL: su nombre y ubicación dependen del contrato de backend. Son opcionales en el DTO
    // para poder leer también el payload del dispositivo tal cual.
    @SerialName("received_at") val receivedAt: String? = null,
    @SerialName("stream_version") val streamVersion: Long? = null,
)
```

Campo JSON	Tipo	Nulo	Regla que aplica la app
`schema_version`	string	No	Solo versión mayor `1`; otra rechaza el frame
`event_id`	UUID	No	Identidad inmutable (`FrameId`); base de la deduplicación
`device_id` · `boot_id`	UUID	No	Dispositivo y sesión de arranque; no autorizan nada
`sequence`	entero ≥ 0	No	Orden de captura dentro del mismo `boot_id`
`captured_at`	RFC 3339 UTC	Sí	Hora del dispositivo; solo es confiable con `clock_quality = synced`
`monotonic_ms`	entero ≥ 0	No	Tiempo relativo desde el arranque
`clock_quality`	string	No	`synced` / `unsynced`; otro valor se trata como no confiable
`mission_id` · `map_version_id`	UUID	Sí	`null` si no hay misión o mapa
`pose`	objeto	Sí	`x_m`, `y_m`, `theta_rad`, `sigma_position_m` (nulo = desconocida, nunca 0), `quality`. Sin pose no hay posición

Campo JSON	Tipo	Nulo	Regla que aplica la app
`readings[]`	lista	No	Un `channel_key` por frame; repetirlo invalida el frame
`readings[].value`	número	Sí	Excluyente con `boolean_value`; ambos presentes invalida el frame
`readings[].boolean_value`	booleano	Sí	`null` nunca se convierte en `false`
`readings[].quality`	string	No	Catálogo de calidad (7.5)
`readings[].calibration_id`	UUID	Sí	Calibración aplicada; el histórico es inmutable
`battery`	objeto	Sí	`voltage_v` ≥ 0, `soc_pct` 0–100, `current_a`, `quality` (método, p. ej. `voltage_only`)
`robot_state` · `motors_active`	string · bool	Sí	Estado reportado (texto original) y orden de motores; no prueban movimiento
`firmware_version`	string	Sí	Versión de firmware
`received_at` · `stream_version`	RFC 3339 · entero	No	Los asigna el backend; el segundo es el cursor de sincronización

Listado 30 · Encabezado de un frame v1.1 (fixture sintético; readings abreviado)
`app/src/test/resources/fixtures/telemetry/frame_v1_1_device_payload.json:1`

```
{
"schema_version": "1.1",
"event_id": "9f225860-42f2-4cd0-8c3a-e4e970395307",
"device_id": "bf8d0f7d-f739-4529-9a8a-5975f5b26960",
"boot_id": "b503cb0b-3866-491b-aebc-e164ea73d9aa",
"sequence": 1843,
"captured_at": "2026-10-11T15:24:12.000Z",
"monotonic_ms": 929450,
"clock_quality": "synced",
"mission_id": "0354909f-cc35-4ebc-822d-bfffeaa80374",
"map_version_id": "ecb3df81-477f-4e72-92c6-8b38f856cdb5",
"pose": {
"x_m": 2.15,
"y_m": 1.40,
"theta_rad": 0.28,
"sigma_position_m": 0.08,
"quality": "estimated"
},
"readings": [
```

7.2 Esquema de Supabase

El frame se reparte en **14 tablas** (script de referencia `docs/backend/esquema-supabase-equipo.sql`). El diagrama marca en naranja las que la app lee; `map_versions` no se consulta desde que el mapa 2D salió de la versión (ADR-009).

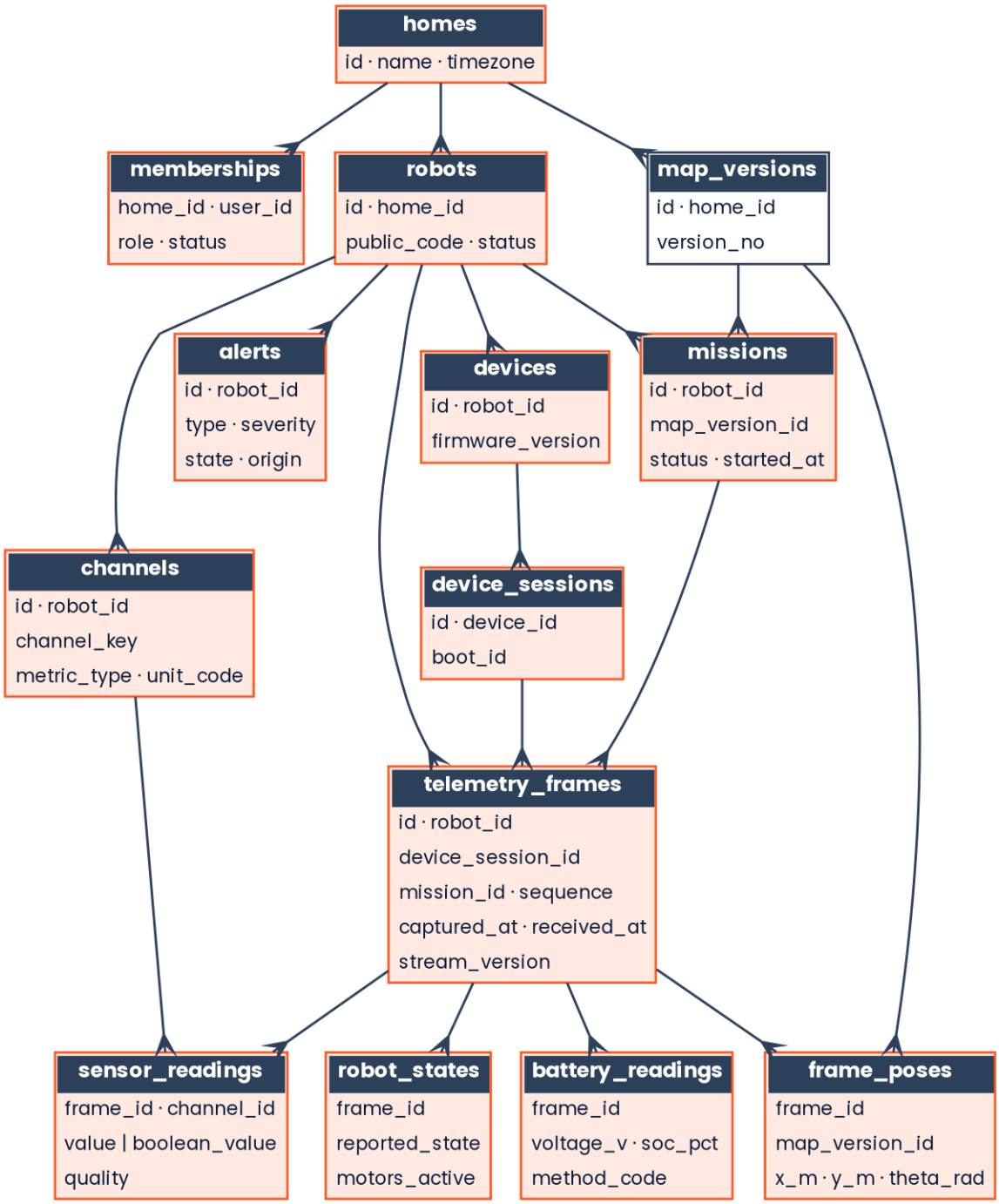


Figura 7. Modelo entidad-relación (patas de gallo en el lado «muchos»). Naranja: tablas que consulta la app.

Las tablas siguientes se generaron a partir del propio script SQL. La columna **App** marca las columnas que la app selecciona.

homes — Domicilio. Define la zona horaria con la que se convierten los filtros de fecha locales a UTC.

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
`id`	uuid	No	✓	PK; por defecto gen_random_uuid()
`name`	text	No	✓	check: length(trim(name)) > 0
`timezone`	text	No	✓	por defecto America/Mexico_City
`created_at`	timestamptz	No		por defecto now()

memberships — Relaciona personas (`auth.users`) con domicilios. Una membresía `active` es lo que habilita la lectura vía RLS.

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
`home_id`	uuid	No	✓	FK → homes.id
`user_id`	uuid	No	✓	FK → users.id
`role`	text	No	✓	check: role in ('reader', 'technician')
`status`	text	No	✓	check: status in ('active', 'inactive')
(tabla)				primary key (home_id, user_id)

robots — Robot de un domicilio. `last_stream_version` lo mantiene el backend; la app usa su propio cursor.

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
`id`	uuid	No	✓	PK; por defecto gen_random_uuid()
`home_id`	uuid	No	✓	FK → homes.id
`public_code`	text	No	✓	único
`last_stream_version`	bigint	No		por defecto 0 check; check: last_stream_version >= 0
`status`	text	No	✓	por defecto active

devices — Dispositivo (microcontrolador) de un robot; guarda la versión de firmware actual.

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
`id`	uuid	No	✓	PK; por defecto gen_random_uuid()
`robot_id`	uuid	No		FK → robots.id
`public_id`	text	No		único
`status`	text	No		por defecto active
`firmware_version`	text	Sí	✓	—

device_sessions — Sesión de arranque (`boot_id`) de un dispositivo; cambia en cada reinicio.

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
`id`	uuid	No		PK; por defecto gen_random_uuid()
`device_id`	uuid	No		FK → devices.id
`boot_id`	uuid	No	✓	—
`started_at`	timestamptz	No		por defecto now()
`clock_quality`	text	No		por defecto unknown
(tabla)				unique (device_id, boot_id)

map_versions — Versión de mapa de un domicilio. La app no la consulta en esta versión (ADR-009).

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
`id`	uuid	No		PK; por defecto gen_random_uuid()
`home_id`	uuid	No		FK → homes.id
`version_no`	int	No		—
`status`	text	No		check: status in ('draft', 'validated', 'retired')
`resolution_m`	numeric	No		por defecto 0.05
`origin_x`	numeric	No		por defecto 0
`origin_y`	numeric	No		por defecto 0
`width`	int	No		—
`height`	int	No		—
`asset_path`	text	No		—
`checksum`	text	No		—
(tabla)				unique (home_id, version_no)

missions — Misión de un robot sobre una versión de mapa.

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
`id`	uuid	No	✓	PK; por defecto gen_random_uuid()
`robot_id`	uuid	No	✓	FK → robots.id
`map_version_id`	uuid	No	✓	FK → map_versions.id
`started_at`	timestamptz	No	✓	por defecto now()
`ended_at`	timestamptz	Sí	✓	—
`status`	text	No	✓	—
`stop_reason`	text	Sí	✓	—

telemetry_frames — Cabecera de cada frame. `stream_version` es el cursor de commit por robot; `id` es el `event_id` del contrato.

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
`id`	uuid	No	✓	PK
`robot_id`	uuid	No	✓	FK → robots.id
`device_session_id`	uuid	No	✓	FK → device_sessions.id
`mission_id`	uuid	Sí	✓	FK → missions.id
`sequence`	bigint	No	✓	check: sequence >= 0
`captured_at`	timestamptz	Sí	✓	—
`received_at`	timestamptz	No	✓	por defecto now()
`monotonic_ms`	bigint	No	✓	check: monotonic_ms >= 0
`clock_quality`	text	No	✓	—
`stream_version`	bigint	No	✓	—
`payload_hash`	text	No		—
(tabla)				unique (device_session_id, sequence)
(tabla)				unique (robot_id, stream_version)

channels — Catálogo de canales de un robot: clave, tipo de métrica y unidad registrada.

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
`id`	uuid	No		PK; por defecto gen_random_uuid()
`robot_id`	uuid	No	✓	FK → robots.id
`channel_key`	text	No	✓	—
`metric_type`	text	No	✓	—
`unit_code`	text	No	✓	—
(tabla)				unique (robot_id, channel_key)

sensor_readings — Una fila por canal y frame. `value` y `boolean_value` son excluyentes; ambos nulos significa «sin dato».

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
`frame_id`	uuid	No	✓	FK → telemetry_frames.id
`channel_id`	uuid	No	✓	FK → channels.id
`value`	double precision	Sí	✓	—
`boolean_value`	boolean	Sí	✓	—
`quality`	text	No	✓	—
`calibration_id`	uuid	Sí	✓	—
(tabla)				primary key (frame_id, channel_id)
(tabla)				check ((value is not null and boolean_value is null) or (value is null and bool

frame_poses — Pose estimada en el frame. Sin fila, el frame no trae posición.

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
frame_id	uuid	No		PK; FK → telemetry_frames.id
map_version_id	uuid	No	✓	FK → map_versions.id
x_m	double precision	No	✓	—
y_m	double precision	No	✓	—
theta_rad	double precision	No	✓	—
sigma_position_m	double precision	Sí	✓	check: sigma_position_m >= 0
sigma_heading_rad	double precision	Sí		check: sigma_heading_rad >= 0
quality	text	No	✓	—

robot_states — Estado reportado por el robot en ese frame.

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
frame_id	uuid	No		PK; FK → telemetry_frames.id
navigation_state	text	No		—
reported_state	text	No	✓	—
motors_active	boolean	Sí	✓	—
wifi_rssi_dbm	int	Sí		—

battery_readings — Batería del frame; `method_code` indica cómo se obtuvo el porcentaje.

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
frame_id	uuid	No		PK; FK → telemetry_frames.id
voltage_v	double precision	Sí	✓	check: voltage_v >= 0
current_a	double precision	Sí	✓	—
soc_pct	numeric	Sí	✓	check: soc_pct >= 0 and soc_pct <= 100
method_code	text	No	✓	—
quality	text	No	✓	—

alerts — Alertas e incidentes generados por la base de datos (reglas del backend o reportadas por el dispositivo).

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
id	uuid	No	✓	PK; por defecto gen_random_uuid()
robot_id	uuid	No	✓	FK → robots.id
type	text	No	✓	—
severity	text	No	✓	check: severity in ('info', 'warning', 'critical')
state	text	No	✓	check: state in ('open', 'acknowledged', 'resolved')
origin	text	No	✓	check: origin in ('device_reported', 'backend_rule')

Columna	Tipo	Nulo	App	Clave, valor por defecto y restricciones
opened_at	timestamptz	No	✓	por defecto now()
last_seen_at	timestamptz	No	✓	por defecto now()
resolved_at	timestamptz	Sí	✓	—

El proyecto vivo difiere del script

El proyecto real de Supabase tiene columnas que el script del 21 de octubre no trae: `telemetry_frames.schema_version` y `raw_payload` (el frame original). La app las usa solo en el build `demo` (sección 10.1). Antes de depender de una columna nueva, contrastarla con el proyecto real y no solo con este script.

7.3 Correspondencia entre el frame y las tablas

Campo del frame	Tabla.columna	Observación
event_id	telemetry_frames.id	
robot_id, mission_id, sequence, captured_at, monotonic_ms, clock_quality	telemetry_frames	
received_at, stream_version	telemetry_frames	Asignados por el backend
schema_version	— (en el proyecto real: telemetry_frames.schema_version)	Si falta se asume 1.1 (STORED_SCHEMA_VERSION)
device_id, firmware_version	devices.id, devices.firmware_version (vía device_sessions)	El firmware es el actual del dispositivo, no el del frame
boot_id	device_sessions.boot_id	
map_version_id, pose.*	frame_poses	Sin fila: sin pose
readings[]	sensor_readings + channels.channel_key	Una fila por canal
battery.*	battery_readings	battery.quality sale de method_code
robot_state, motors_active	robot_states.reported_state, .motors_active	

7.4 Tipos del dominio

El dominio no usa `String` ni `UUID` desnudos para las identidades: cada una es una *value class* para que el compilador impida mezclarlas. Los catálogos abiertos se modelan con interfaces selladas que **conservan el texto original** cuando el valor no es conocido (`Unknown(raw)`).

Tipo	Forma	Notas
FrameId, DeviceId, BootId, CalibrationId, MissionId, MapVersionId, HomeId, RobotId, AlertId	@JvmInline value class sobre UUID	Sin costo en ejecución; UserId envuelve String

Tipo	Forma	Notas
`StreamVersion`	value class sobre `Long`	Comparable; `NONE` y `next()` para el cursor
`ChannelKey`	value class sobre `String`	No hay lista fija de canales
`TelemetryFrame`	`data class` con invariantes en `init`	`sequence ≥ 0`, `monotonicMs ≥ 0`, un `channel_key` por frame; `reliableCapturedAt`
`FrameRecord`	`Interpreted(frame)` | `Rejected(version, id, reason)`	Resultado del mapeador; el rechazo no detiene el cursor
`ReadingValue`	`Numeric` (finito) | `BooleanValue` | `Absent`	`Absent` nunca equivale a 0 ni a `false`
`ReadingQuality`	sellada con `Unknown(raw)`	Catálogo en 7.5
`ReadingInterpretation`	`Measured`, `RawSignal`, `NotInstalled`, `Warming`, `Fault`, `OutOfRange`, `NoData`, `Uncertain`	Salida de `ReadingInterpreter`
`Pose`	`xM`, `yM`, `thetaRad`, `sigmaPositionM?`, `quality`	Valores finitos; incertidumbre ≥ 0
`BatteryInterpretation`	`WithPercentage` | `PercentageUnavailable` | `NoData`	La tensión nunca se convierte en porcentaje
`SessionState`	`Initializing`, `Authenticated`, `NotAuthenticated`, `RefreshFailed`	
`HomeScope`	`Home` + lista de `Robot`	`contains(robotId)`
`SyncStatus`, `TelemetrySyncState`	Ver sección 6.5	Estado observable del sincronizador

7.5 Catálogos

Catálogo	Valores conocidos	Valor desconocido
`readings[].quality`	`valid`, `estimated`, `uncalibrated`, `warming`, `fault`, `out_of_range`, `not_installed`	`ReadingQuality.Unknown` → `Uncertain`
`clock_quality`	`synced`, `unsynced`	`Unknown` → reloj no confiable
`alerts.state`	`open`, `acknowledged`, `resolved`	`AlertState.Unknown(raw)`
`alerts.origin`	`device_reported`, `backend_rule`	`AlertOrigin.Unknown(raw)`
`alerts.severity`	`info`, `warning`, `critical` (check de la tabla)	Texto sin interpretar
`missions.status`	`running`, `completed`, `aborted` (catálogo no publicado)	`MissionStatus.Unknown(raw)`
`memberships.status`	`active`, `inactive`	`MembershipStatus.Unknown(raw)`
`memberships.role`	`reader`, `technician` (check de la tabla)	Se conserva el texto; no se autoriza por rol
`robot_state`, `pose.quality`, `battery.quality`	Texto libre (p. ej. `moving`, `estimated`, `voltage_only`)	Se muestra el texto original

Interpretación de la calidad de lectura (`ReadingInterpreter`):

quality	Resultado en la app
`valid`, `estimated`	`Measured`: valor con la unidad del catálogo
`uncalibrated`	`RawSignal`: señal cruda; **nunca** lleva unidad de concentración (`ppm`)
`warming`	`Warming`: no se muestra concentración
`fault`, `out_of_range`	`Fault`, `OutOfRange`
`not_installed`	`NotInstalled`: estado propio, nunca cero
valor `null` con calidad válida, estimada o sin calibrar	`NoData` («Sin dato»)
otra calidad	`Uncertain`: se conserva el valor, no se afirma su validez

7.6 Almacenamiento local

La app **no tiene base de datos local**. Lo único que persiste el código propio es un archivo de `SharedPreferences` (`centinela_ui`):

Clave	Contenido	Notas
`theme`	`System` | `Light` | `Dark`	Tema elegido en Ajustes
`last_email`	Último correo con el que se inició sesión	Rellena el formulario y deriva el nombre del saludo; **no** se guarda la contraseña

Los tokens de sesión los almacena y renueva el SDK de Supabase; la aplicación no los lee ni los copia. `android:allowBackup="false"` evita que el sistema respalde estos archivos.

7.7 Ejemplo de consulta

Listado 31 · Equivalente HTTP de framesAfter (valores ficticios; el SDK arma la solicitud)

```
GET https://<proyecto>.supabase.co/rest/v1/telemetry_frames
    ?select=id,robot_id,mission_id,sequence,captured_at,received_at,monotonic_ms,clock_quality,stream_ve
        rsion,
            device_sessions(boot_id,devices(id,firmware_version)),
            sensor_readings(value,boolean_value,quality,calibration_id,channels(channel_key)),
            frame_poses(...),robot_states(...),battery_readings(...)
    &robot_id=eq.<uuid-del-robot>
    &stream_version=gt.1842
    &order=stream_version.asc
    &limit=100
apikey: <clave publicable>          Authorization: Bearer <token de la sesión>
```

8. Reglas de dominio

Las reglas viven en `:core`, sin Android ni red, y se prueban en JVM. Cada una existe para impedir un error de interpretación que, en un monitor de gases, tendría consecuencias reales: tomar una señal cruda por una concentración, un sensor ausente por aire limpio o un dato viejo por actual.

8.1 Interpretación de lecturas

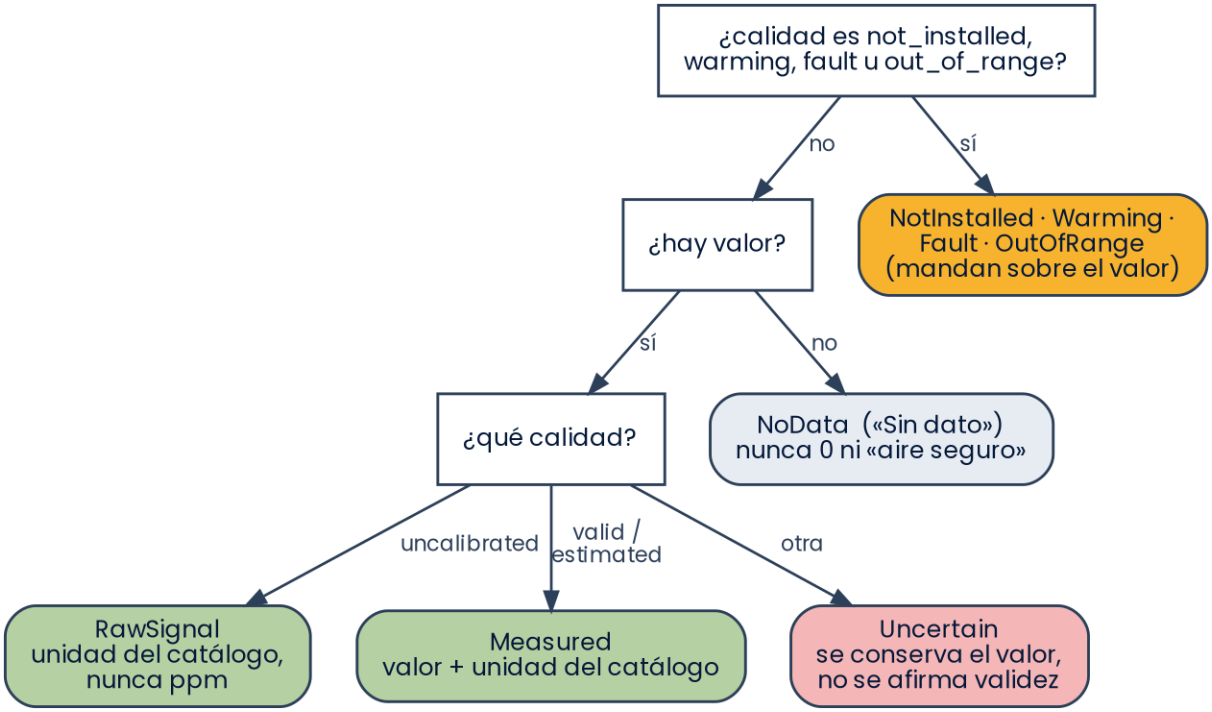


Figura 8. Precedencia de ReadingInterpreter.interpret.

Listado 32 · ReadingInterpreter.interpret `core/src/main/kotlin/com/robotcentinela/app/environmentalmonitoring/domain/ReadingInterpretation.kt:70`

```
fun interpret(reading: SensorReading, capability: ChannelCapability? = null): ReadingInterpretation {
    val quality = reading.quality
    when (quality) {
        ReadingQuality.NotInstalled -> return ReadingInterpretation.NotInstalled
        ReadingQuality.Warming -> return ReadingInterpretation.Warming
        ReadingQuality.Fault -> return ReadingInterpretation.Fault
        ReadingQuality.OutOfRange -> return ReadingInterpretation.OutOfRange
        else -> Unit
    }

    if (reading.value == ReadingValue.Absent) return ReadingInterpretation.NoData

    val unitCode = capability?.unitCode
    return when (quality) {
        ReadingQuality.Uncalibrated -> ReadingInterpretation.RawSignal(
            value = reading.value,
            unitCode = unitCode?.takeUnless { it in CONCENTRATION_UNITS },
        )
        ReadingQuality.Valid, ReadingQuality.Estimated ->
            ReadingInterpretation.Measured(reading.value, unitCode, quality)
        else -> ReadingInterpretation.Uncertain(reading.value, quality.raw)
    }
}
```

- **Estados del sensor mandan** sobre cualquier valor: un sensor calentando o con fallo no muestra concentración aunque envíe un número.
- **Señal cruda ≠ concentración.** Una lectura `uncalibrated` conserva su valor pero se descarta la unidad si el catálogo declara `ppm` (`CONCENTRATION_UNITS`).
- **La unidad viene del catálogo** (`channels.unit_code`), nunca del frame: «la unidad no se acepta libremente del emisor».

8.2 Frescura de los datos

Cada canal tiene su propia antigüedad. Un canal pasa a **«atrasado»** cuando su última muestra supera `stalePeriods` (3) periodos esperados (2 s por defecto), aunque otros canales sigan activos. El instante de referencia es `captured_at` **si el reloj estaba sincronizado**; si no, `received_at` como aproximación explícita.

Listado 33 · FreshnessPolicy.evaluate `core/src/main/kotlin/com/robotcentinela/app/environmentalmonitoring/domain/FreshnessPolicy.kt:46`

```
fun evaluate(sampleInstant: Instant, expectedPeriod: Duration?, now: Instant): DataFreshness {
    // Si el reloj del teléfono va atrasado respecto a la muestra, la edad queda en cero en
    // lugar de ser negativa.
    val age = java.time.Duration.between(sampleInstant, now).toKotlinDuration()
        .coerceAtLeast(Duration.ZERO)
    val limit = (expectedPeriod?.takeIf { it.isPositive() } ?: defaultExpectedPeriod) * stalePeriods
    return if (age > limit) DataFreshness.Stale(age, limit) else DataFreshness.Fresh(age)
}
```

Listado 34 · CaptureStamp.isNewerThan: qué muestra es más reciente
`core/src/main/kotlin/com/robotcentinela/app/telemetry/domain/LatestTelemetryProjection.kt:32`

```
fun isNewerThan(other: CaptureStamp): Boolean {
    if (deviceId == other.deviceId && bootId == other.bootId) {
        return sequence > other.sequence
    }
    val mine = reliableCapturedAt
    val theirs = other.reliableCapturedAt
    val comparison = if (mine != null && theirs != null) {
        mine.compareTo(theirs)
    } else {
        receivedAt.compareTo(other.receivedAt)
    }
    return comparison > 0 || (comparison == 0 && streamVersion > other.streamVersion)
}
```

8.3 Proyección del último dato

`LatestTelemetryProjection` es **inmutable** y guarda, por canal, la última muestra con su propia marca de captura. Una muestra retrasada (por ejemplo, un frame enviado tras una desconexión) entra al historial pero **no reemplaza** un dato más reciente: el tablero nunca retrocede.

Listado 35 · LatestTelemetryProjection.apply
`core/src/main/kotlin/com/robotcentinela/app/telemetry/domain/LatestTelemetryProjection.kt:84`

```
fun apply(frame: TelemetryFrame): LatestTelemetryProjection {
    val stamp = CaptureStamp.of(frame)
```

```
    val updatedChannels = channels.toMutableMap()
    for (reading in frame.readings) {
        val current = updatedChannels[reading.channelKey]
        if (current == null || stamp.isNewerThan(current.stamp)) {
            updatedChannels[reading.channelKey] = ChannelSample(reading, stamp)
        }
    }

    val currentLatest = latestFrame
    val newLatest = if (currentLatest == null || stamp.isNewerThan(CaptureStamp.of(currentLatest))) {
        frame
    } else {
        currentLatest
    }

    val previousReceived = lastReceivedAt
    val newLastReceived = if (previousReceived == null || frame.receivedAt > previousReceived) {
        frame.receivedAt
    } else {
        previousReceived
    }

    return LatestTelemetryProjection(updatedChannels, newLatest, newLastReceived)
}
```

8.4 Salud de la ingesta y del robot

`IngestConnectivityPolicy` compara el `received_at` más reciente (asignado por el servidor) con la hora del teléfono; pasados 10 s sin frames, la ingesta se declara *interrumpida*. **No** se afirma que el robot esté apagado: el corte puede estar en el puente, el broker o la red. Si los relojes difieren mucho, el resultado se desplaza.

Listado 36 · IngestConnectivityPolicy.evaluate
`core/src/main/kotlin/com/robotcentinela/app/robothealth/domain/IngestConnectivityPolicy.kt:39`

```
fun evaluate(lastReceivedAt: Instant?, now: Instant): IngestStatus {
    if (lastReceivedAt == null) return IngestStatus.NoDataYet
    val since = java.time.Duration.between(lastReceivedAt, now).toKotlinDuration()
        .coerceAtLeast(Duration.ZERO)
    return if (since > offlineAfter) {
        IngestStatus.Interrupted(lastReceivedAt, since)
    } else {
        IngestStatus.Receiving(lastReceivedAt, since)
    }
}
```

Listado 37 · BatteryInterpretation.of: nunca se infiere porcentaje desde la tensión
`core/src/main/kotlin/com/robotcentinela/app/robothealth/domain/BatteryStatus.kt:46`

```
fun of(status: BatteryStatus?): BatteryInterpretation {
    if (status == null) return NoData
    val voltage = status.voltageV?.takeIf { it.isFinite() && it >= 0 }
    val soc = status.socPct?.takeIf { it.isFinite() && it in 0.0..100.0 }
    return when {
        soc != null -> WithPercentage(voltage, soc, status.quality)
        voltage != null -> PercentageUnavailable(voltage, status.quality)
        else -> NoData
    }
}
```

8.5 Resumen de reglas

Regla	Implementación	Parámetro
Señal sin calibrar nunca es ppm	`ReadingInterpreter`	`CONCENTRATION_UNITS = {ppm}`
`null` no es 0 ni `false`	`ReadingValue.Absent`, mapeador	—
Canal atrasado	`FreshnessPolicy`	2 s × 3 periodos
Ingesta interrumpida	`IngestConnectivityPolicy`	10 s
El último dato no retrocede	`LatestTelemetryProjection` + `CaptureStamp`	orden por `sequence`/`captured_at`/`received_at`
Hora no confiable se declara aproximada	`TelemetryFrame.reliableCapturedAt`	`clock_quality = synced`
Porcentaje de batería solo si el dispositivo lo reporta	`BatteryInterpretation.of`	SOC 0–100, tensión ≥ 0
La app no autoriza por rol ni calcula alertas	`:core` no contiene umbrales	—

9. Capa de presentación

9.1 Estructura de la interfaz

La UI es **Jetpack Compose** con Material 3 sobre un tema propio. Hay una sola `Activity` (`MainActivity`) que dibuja de borde a borde, instala el splash de la marca y delega todo en `CentinelaApp`. Cada pantalla tiene un `ViewModel` que expone un único estado inmutable.

Paquete ui/	Contenido
`app`	`CentinelaApp`, `RootViewModel` (`RootState`), `UiPreferences`, `UiDependencies`
`auth`	`LoginScreen`, `LoginViewModel` (validación de correo y mensajes de error traducidos)
`homes`	Selección explícita de domicilio
`main`	`MainScreen` (navegación por pestañas), `MainViewModel` (estado en vivo compartido)
`summary` · `map` · `missions` · `history` · `events` · `settings`	Pantallas y hojas; `map` solo muestra «No disponible / Implementación futura»
`components`	Tarjetas de métrica, gráfica de líneas en `Canvas`, controles, marca (`VisorSplash`, `VisorMark`), `FutureFeatureNotice`
`model`	Presentación sin Compose: etiquetas de estado, formatos, rangos de fechas, modelos de gráfica y tarjetas
`theme`	`CentinelaColors`, `VisorPalette`, `Theme`, `Type`, `ThemePreference`

9.2 Estado y navegación

`MainScreen` crea su `MainViewModel` con clave por domicilio y arranca la sincronización con el ciclo de vida. Las pestañas (Resumen, Mapa, Misiones, Historial) y Eventos usan **rutas tipadas** de Navigation: cada ruta es un `@Serializable data object`. Con ancho ≥ 840 dp la barra inferior se sustituye por un riel lateral.

Listado 38 · Rutas tipadas y ciclo de vida de la sincronización
`app/src/main/kotlin/com/robotcentinela/app/ui/main/MainScreen.kt:84`

```
@Serializable data object SummaryRoute

@Serializable data object MapRoute

@Serializable data object MissionsRoute

@Serializable data object HistoryRoute

@Serializable data object EventsRoute


/** Destinos de la barra de navegación (mockup móvil: Resumen, Mapa, Misiones e Historial). Eventos se
    abre con la campana. */
private enum class Tab(val label: String, val icon: Int, val route: Any) {
    Summary("Resumen", R.drawable.ic_assignment, SummaryRoute),
    MapTab("Mapa", R.drawable.ic_map, MapRoute),
    Missions("Misiones", R.drawable.ic_edit_road, MissionsRoute),
    History("Historial", R.drawable.ic_history, HistoryRoute),
}

@Composable
fun MainScreen(scope: HomeScope, onChangeHome: () -> Unit) {
    val deps = uiDependencies
    val vm: MainViewModel = viewModel(key = "main-${scope.home.id.value}") {
        MainViewModel(deps.container, scope) }
    val live by vm.state.collectAsStateWithLifecycle()
    val colors = CentinelaTheme.colors

    // La sincronización solo corre con la pantalla visible (RB28).
    LifecycleStartEffect(vm) {
        vm.onStart()
        onStopOrDispose { vm.onStop() }
    }

    val navController = rememberNavController()
```

Listado 39 · LiveUiState: combine de cuatro orígenes con stateIn
`app/src/main/kotlin/com/robotcentinela/app/ui/main/MainViewModel.kt:145`

```
val state: StateFlow<LiveUiState> = combine(core, extras, alertExtras, selectedRobot) { c, e, a, robot
    ->
    buildState(robot, c, e, a)
}.stateIn(
    viewModelScope,
    SharingStarted.WhileSubscribed(5_000),
    LiveUiState(robot = selectedRobot.value, robots = scope.robots, zone = zone, today = today(),
        alertRange = alertRange.value),
)
```

- **Un solo `LiveUiState`** alimenta Resumen y el aviso persistente de alertas; las pantallas de historial y eventos tienen su propio ViewModel.

- **Sin lógica de negocio en los composables:** las decisiones (qué etiqueta, qué tono, qué unidad) se toman en `ui/model` y en `:core`, y son probables sin Compose.
- `collectAsStateWithLifecycle()` en todas las pantallas, para no recomponer ni consultar con la app en segundo plano.

9.3 Tema e identidad

La identidad V.I.S.O.R. (ADR-008) se concentra en `ui/theme`. Las pantallas **no** usan hexadecimales: consumen tokens (`CentinelaTheme.colors`). Los tokens oficiales están en `VisorPalette`; los derivados (`on*`, `criticalText`, `statusNeutral`) existen para cumplir el contraste de la guía (4.5:1 en texto y 3:1 en gráficos).

Color	Hex	Uso
Black Blue	`#041839`	Fondo oscuro, splash, texto principal en claro
Off White	`#EEEEEE`	Fondo claro
Navy	`#2D4059`	Cabecera, barra de navegación, primario
Bright Orange	`#FF5722`	Acento, foco, avisos y «atrasado»
Validado	`#B5D0A2`	Estado operativo / dato válido
Alerta	`#F7B32D`	Lecturas de riesgo (relleno con texto oscuro)
Crítico	`#C00000`	Lecturas críticas; en oscuro, el texto usa `criticalText` más claro

Listado 40 · CentinelaTheme: tema claro/oscuro con colores animados
`app/src/main/kotlin/com/robotcentinela/app/ui/theme/Theme.kt:134`

```
fun CentinelaTheme(
    preference: ThemePreference = ThemePreference.System,
    content: @Composable () -> Unit,
) {
    val dark = when (preference) {
        ThemePreference.System -> isSystemInDarkTheme()
        ThemePreference.Light -> false
        ThemePreference.Dark -> true
    }
    val targetColors = if (dark) DarkCentinelaColors else LightCentinelaColors
    val colors = animateCentinelaColors(targetColors)
    CompositionLocalProvider(LocalCentinelaColors provides colors) {
        MaterialTheme(
            colorScheme = colors.toMaterial(),
            typography = CentinelaTypography,
            shapes = Shapes(
                extraSmall = CentinelaShapes.control,
                small = CentinelaShapes.control,
                medium = CentinelaShapes.card,
                large = CentinelaShapes.card,
                extraLarge = CentinelaShapes.sheet,
            ),
            content = content,
        )
    }
}
```

- **Tipografía:** Poppins en cuerpo (variante con **dígitos tabulares**, porque la original no trae `tnum` y los valores en vivo no deben «bailar») y Questrial, solo peso regular, en titulares; no se pide negrita para no sintetizar una deformación.
- **Íconos:** 20 vectores de la marca sustituyen a los equivalentes de Material por nombre; el resto sigue siendo Material Symbols. Todos son `VectorDrawable` (80 en `res/drawable`).
- **Splash:** `core-splashscreen` con `Theme.Centinela.Starting`; la pantalla de carga de Compose (`VisorSplash`) reproduce el diseño de la marca.

9.4 Adaptación a pantallas

La interfaz se adapta por **ancho disponible** (`BoxWithConstraints`), no por tipo de dispositivo, y respeta la escala de fuente del sistema.

Ancho	Efecto
≥ 560 dp	Misiones: el botón «Generar nueva misión» va junto al título; Historial pasa a disposición ancha
≥ 600 dp	Resumen: tarjetas de métrica en 4 columnas (2 en estrecho)
≥ 760 dp	Resumen y Misiones: disposición ancha
≥ 840 dp	Navegación: riel lateral en lugar de barra inferior

Las tarjetas de métrica admiten dos líneas de título para que la escala de fuente 1.3–2.0 no corte el texto.

9.5 Funciones fuera de esta versión

`FutureFeatureNotice` muestra «No disponible · Implementación futura» donde antes había un mapa 2D, rutas o lecturas por habitación (ADR-009). La pose y el `map_version_id` siguen llegando en el frame y viven en el dominio, pero no se dibujan. «Guardar» en Nueva misión no envía nada (R9).

10. Compilación, variantes y distribución

10.1 Variantes (buildTypes)

Listado 41 · app/build.gradle.kts: buildTypes `app/build.gradle.kts:50`

```
buildTypes {
    debug {
        buildConfigField("String", "DEV_EMAIL", javaString(localProperty("centinela.devEmail")))
        buildConfigField("String", "DEV_PASSWORD", javaString(localProperty("centinela.devPassword")))
    }
    release {
        buildConfigField("String", "DEV_EMAIL", "\"\"")
        buildConfigField("String", "DEV_PASSWORD", "\"\"")
        // Sin keystore del equipo todavía: se firma con la clave de depuración de Android para poder
        // instalarlo. Sirve para pruebas y demostraciones, no para publicar en una tienda.
        signingConfig = signingConfigs.getByName("debug")
        // R8: reduce el código y los recursos (el APK sin minificar pesaba 34 MB).
        isMinifyEnabled = true
        isShrinkResources = true
        proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")
    }
    // Demostración beta (ADR-007): lee de Supabase como el rol `anon`, sin pantalla de login ni cuenta
    // embebida. Se firma con la clave de depuración de Android: sirve para instalar, no para publicar.
    create("demo") {
        initWith(getByName("release"))
        matchingFallbacks += "release"
        signingConfig = signingConfigs.getByName("debug")
        versionNameSuffix = "-demo"
        buildConfigField("boolean", "DEMO_PUBLIC_READ", "true")
    }
}
```

Variante	Sesión	Minificación	Firma	BuildConfig relevante
`debug`	Supabase Auth	No	Clave de depuración	`DEV_EMAIL`, `DEV_PASSWORD` desde `local.properties`
`release`	Supabase Auth	R8 + recorte de recursos	Clave de depuración (temporal)	`DEV_*` vacíos
`demo`	Sesión fija de lectura pública (rol `anon`)	R8 + recorte de recursos	Clave de depuración	`DEMO_PUBLIC_READ = true`; `versionNameSuffix = "-demo"`

El build **`demo`** es temporal (ADR-007): sustituye únicamente `AuthPort` y `HomeScopePort` por `PublicReadAuthAdapter` y `PublicReadHomeScopeAdapter`, y pide además `schema_version` y `raw_payload` para obtener `boot_id` y `device_id` cuando RLS oculta `device_sessions`. `:core` no se entera. Se retira cuando exista un usuario con membresía (B21).

10.2 Compilar y probar

Listado 42 · Comandos habituales

```
export JAVA_HOME=/ruta/a/jdk-21 ANDROID_HOME=$HOME/Android/Sdk
./gradlew :core:test            --no-daemon --max-workers=1
./gradlew :app:testDebugUnitTest --no-daemon --max-workers=1
./gradlew assembleDebug         --no-daemon --max-workers=1   # app/build/outputs/apk/debug
./gradlew assembleDemo          --no-daemon --max-workers=1   # app/build/outputs/apk/demo
./gradlew assembleRelease       --no-daemon --max-workers=1   # app/build/outputs/apk/release
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

Los tres APK comparten `applicationId`: **instalar uno reemplaza al otro** y borra la sesión y los ajustes locales.

10.3 R8

`release` y `demo` se minimizan con R8. Las reglas de `app/proguard-rules.pro` protegen lo que `kotlinx.serialization` necesita por reflexión (companions y serializadores de las clases propias) y silencian las referencias de Ktor/supabase-kt a APIs de JVM de escritorio que no existen en Android.

Listado 43 · Reglas de R8 `app/proguard-rules.pro:5`

```
-keepattributes *Annotation*, InnerClasses, Signature, RuntimeVisibleAnnotations
-keepclassmembers class com.robotcentinela.app.** {
    *** Companion;
}
-keepclasseswithmembers class com.robotcentinela.app.** {
    kotlinx.serialization.KSerializer serializer(...);
}
-keep,includedescriptorclasses class com.robotcentinela.app.**$$serializer { *; }

# Ktor y supabase-kt referencian SLF4J y APIs de JVM de escritorio que no existen en Android.
-dontwarn org.slf4j.**
-dontwarn java.lang.management.**
-dontwarn javax.naming.**
-dontwarn org.slf4j.impl.StaticLoggerBinder
```

Medida (Samsung Galaxy A30, Android 11)	Sin R8	Con R8
Tamaño del APK	34.5 MB	5.3 MB
Memoria total (PSS) tras recorrer la app	121.6 MB	92.5 MB
Heap de Java	17.7 MB	9.8 MB
Arranque en frío	1.48 s	1.11 s
Cuadros lentos (`gfxinfo`)	26.9 %	25.8 %

Si R8 rompe algo

Los síntomas típicos son `SerializationException` en el primer `decodeList` o un `ClassNotFoundException` de Ktor. Compare con el build debug (sin R8) y revise `mapping.txt` antes de ampliar las reglas `-keep`.

10.4 Firma, versionado y publicación

- **Versionado:** `versionCode` entero creciente y `versionName` con sufijo `-beta`; el APK demo añade `-demo`. Se cambian juntos en `app/build.gradle.kts`.
- **Firma:** los builds `release` y `demo` se firman con la clave de depuración de Android (`CN=Android Debug`). Sirve para instalar y mostrar; **no** para una tienda. Un keystore propio nunca se versiona (`*.jks` y `*.keystore` están en `.gitignore`).
- **Artefactos:** los APK se publican como *pre-release* de GitHub con su SHA-256 y el manual de usuario en PDF.
- **Verificación del APK:** `aapt2 dump badging` (paquete, versión, `minSdk`), `apksigner verify --print-certs` y una búsqueda de secretos.

Listado 44 · Comprobaciones previas a publicar

```
$ANDROID_HOME/build-tools/36.0.0/aapt2 dump badging app-release.apk | grep -E "package|sdkVersion"
$ANDROID_HOME/build-tools/36.0.0/apksigner verify --print-certs app-release.apk
unzip -p app-release.apk 'classes*.dex' | strings | grep -c "sb_secret_"     # debe ser 0
sha256sum RobotCentinela-0.5.0-beta.apk
```

11. Pruebas y calidad

11.1 Estrategia

La mayor parte de la lógica está en `:core`, que se prueba en **JVM puro**, sin Android, red ni emulador. Los adaptadores se prueban contra **fixtures JSON sintéticos** (nunca datos reales) y la interfaz se verifica a mano en un teléfono físico. Las pruebas del sincronizador usan un consultador y un notificador falsos, y `runTest` con tiempo virtual (`advanceTimeBy`), de modo que los 30 s de reconciliación no cuestan tiempo real.

Clase	Módulo	Pruebas	Qué verifica
`TelemetrySynchronizerTest`	core	12	Snapshot, deltas paginados, ráfaga de 200 avisos, duplicados, interrupción, hueco de retención, reconciliación, rechazados, error, parada
`ReadingInterpretationTest`	core	10	Precedencia de estados, señal cruda sin ppm, `null` ≠ 0, calidad desconocida
`RobotHealthDomainTest`	core	7	Batería (solo voltaje, SOC fuera de rango) y umbral de ingesta de 10 s
`HomeScopeUseCasesTest`	core	5	Solo membresías activas, robots fuera del ámbito y cierre de sesión
`LatestTelemetryProjectionTest`	core	5	Una muestra atrasada no reemplaza a una reciente; orden entre arranques y con reloj no sincronizado
`FreshnessPolicyTest`	core	4	Límite de tres periodos, periodo del catálogo y edad nunca negativa

Clase	Módulo	Pruebas	Qué verifica
`ProvisionalContractsTest`	core	4	Resumen de misión sin inventar ceros y misiones de robots ajenos
`UtcIntervalTest`	core	4	Fechas locales a intervalo UTC semiabierto
`DescribeChannelStatusTest`	core	3	Interpretación y edad por canal; canal del catálogo sin reporte como «Sin dato»
`QueryAlertHistoryTest`	core	3	Filtro local a UTC con la zona del domicilio y robots ajenos
`TelemetryFrameMapperTest`	app	12	Contrato v1.1, versión mayor no soportada, `value` y `boolean_value` a la vez, `channel_key` repetido, campos aditivos
`PresentationTest`	app	12	Señal cruda en ADC y nunca en ppm, «Sin dato», atraso y rangos de fechas (sin Compose)
`NormalizedFrameAssemblerTest`	app	7	Reconstrucción desde tablas normalizadas, relaciones como arreglo y respaldo con `raw_payload`
`SupabaseConfigTest`	app	6	URL HTTPS, claves privilegiadas rechazadas, `toString` sin clave
`PublicReadAuthAdapterTest`	app	3	Sesión fija de la demo

Total: **97 pruebas** (57 en `:core` y 40 en `:app`), todas en verde en la versión 0.5.0-beta.

Listado 45 · Prueba de la ráfaga: el buffer descarta avisos, pero el cursor no pierde frames
`core/src/test/kotlin/com/robotcentinela/app/telemetry/application/TelemetrySynchronizerTest.kt:133`

```
fun `una rafaga de avisos que desborda el buffer no pierde frames`() = runTest {
    query.persist(1)
    val sync = startedSynchronizer()
    notice(TelemetryNotice.Subscribed)

    // Mientras la consulta está en curso llegan muchos más avisos que la capacidad del buffer (64).
    query.latency = 5.seconds
    query.persist(2)
    notice(TelemetryNotice.FrameCommitted)
    repeat(200) {
        query.persist(3L + it)
        notice(TelemetryNotice.FrameCommitted)
    }
    advanceTimeBy(30.seconds)
    runCurrent()

    // Los avisos descartados eran redundantes: el cursor por stream_version recupera todo.
    assertEquals(StreamVersion(202), sync.state.value.lastAppliedVersion)
    assertEquals(SyncStatus.Live, sync.state.value.status)
}
```

Listado 46 · Prueba de una regla semántica (la señal cruda no lleva ppm)
`core/src/test/kotlin/com/robotcentinela/app/environmentalmonitoring/domain/ReadingInterpretationTest.kt:31`

```
fun `senal sin calibrar nunca lleva unidad de concentracion`() {
    val result = ReadingInterpreter.interpret(
        reading("mq6_raw", 1860.0, ReadingQuality.Uncalibrated),
        capability("mq6_raw", "ppm"),
    )
    val raw = assertIs<ReadingInterpretation.RawSignal>(result)
    assertNull(raw.unitCode)
}
```

11.2 Fixtures

Archivo (app/src/test/resources/fixtures/telemetry/)	Para qué
`frame_v1_1_device_payload.json`	Copia del contrato tal como lo publica el dispositivo; verifica que el DTO lo replica
`frame_qa_sensor_states.json`	`null`, `warming`, `fault`, `out_of_range`, `not_installed`, calidad fuera de catálogo, reloj `unsynced`
`frame_qa_invalid_both_values.json`	`value` y `boolean_value` a la vez: debe rechazarse
`frame_qa_schema_v2.json`	`schema_version` 2.0: debe rechazarse
`frame_supabase_embebido.json`	Frame como lo devuelve PostgREST desde el esquema normalizado; prueba `NormalizedFrameAssembler`

11.3 Pruebas en dispositivo

No se usa emulador. Cada release se probó por ADB en un Samsung Galaxy A30 (Android 11, 2.8 GB de RAM): recorrido completo en tema claro y oscuro, login con credenciales **falsas** (el error debe llegar de Supabase Auth), y una matriz de pantalla que cambia temporalmente `wm size`, `wm density`, `font_scale`, rotación y modo noche. **Esos ajustes se restauran siempre**, incluso ante un fallo.

Listado 47 · Matriz de pantalla y medición (con restauración final)

```
adb shell wm size 720x1280 ; adb shell wm density 320           # teléfono chico
adb shell settings put system font_scale 2.0                     # accesibilidad
adb shell cmd uimode night yes                                   # tema oscuro
adb exec-out screencap -p > captura.png
adb shell dumpsys meminfo com.robotcentinela.app                 # PSS y heap
adb shell dumpsys gfxinfo com.robotcentinela.app                 # cuadros lentos
adb shell wm size reset ; adb shell wm density reset ; adb shell settings put system font_scale 1.0
```

> **Con ADB inalámbrico**
> No active el modo avión ni apague el Wi-Fi durante las pruebas: se cortaría el propio enlace de depuración.

11.4 Lo que no está cubierto

- **No hay pruebas instrumentadas ni de interfaz automatizadas**: la UI (Compose) se verifica a mano; `PresentationTest` cubre solo la capa `ui/model`.
- **No hay prueba de integración contra Supabase**: los adaptadores se prueban con fixtures; Realtime y el flujo de login válido se probaron manualmente hasta donde el proyecto lo permite (B21).
- No hay análisis estático (detekt/ktlint) ni cobertura medida en el pipeline; el formato sigue `kotlin.code.style=official`.

12. Compatibilidad y limitaciones

12.1 Plataforma y dispositivos

Aspecto	Valor / estado
Versión mínima de Android	10 (API 29). `minSdk = 29`
`targetSdk` / `compileSdk`	36 / 37 (Compose 1.12, Lifecycle 2.11 y Navigation 2.10 se publican compilados con API 37)
Verificado en dispositivo	Samsung SM-A305G (Galaxy A30), Android 11, 1080×2340 @ 420 dpi, 2.8 GB de RAM. Otras versiones se asumen por compatibilidad de API, **no** se han probado
Arquitecturas	Sin código nativo: cualquier ABI que soporte el sistema
Orientación y tamaños	Vertical, horizontal y tableta (1600×2560) revisados por emulación de pantalla (`wm size`), no en una tableta física
Accesibilidad	Escala de fuente 1.0 a 2.0 revisada; formulario de inicio de sesión con objetivos táctiles de 48 dp (ADR-006); contraste 4.5:1 en texto según la guía. No se auditó con TalkBack
Idioma	Solo español (cadenas en `strings.xml` y literales en `ui/model`); sin internacionalización
Firma	Clave de depuración: sirve para instalar, no para tiendas ni para actualizar sobre otra firma

12.2 Contrato y backend

- **Solo `schema_version` 1.x.** Una versión mayor distinta rechaza el frame (se cuenta, no se muestra). Soportar v2 requiere un adaptador nuevo (sección 15).
- **El proyecto vivo no coincide con el script del equipo** (columnas extra, políticas distintas). El código se ajusta al proyecto real; los documentos se contrastan antes de confiar en ellos.
- **B19 — lectura anónima:** el rol `anon` puede leer telemetría con la clave pública que va dentro del APK. El build `demo` depende de ello y dejará de mostrar datos cuando backend cierre la lectura (comportamiento correcto).
- **B21 — sin usuario con membresía:** el flujo de inicio de sesión válido no se ha podido probar de extremo a extremo; la solicitud está en `docs/backend/solicitud-usuario-demo.md`.
- **Datos actuales escasos:** pocos frames y el último con semanas de antigüedad, por lo que los canales aparecen «Atrasado».
- **Alertas y misiones:** la lógica vive en la base de datos; `alert_occurrences` y la vista `telemetry_frames_v1` no existen en el proyecto real (B22). Misiones no tiene nombre, estaciones ni energía (B16).

12.3 Sincronización y datos

- **Snapshot de un solo frame:** el expediente pide un snapshot con marca de agua en una transacción; una consulta PostgREST de un recurso lo cumple, pero un snapshot compuesto exigiría una RPC (B3).
- **Sin caché ni modo sin conexión persistente:** al perder la red se muestra lo que ya está en memoria; al cerrar la app se pierde. No hay Room ni almacenamiento de frames.
- **Sin push:** las alertas se consultan cada 30 s mientras la app está visible; no hay notificaciones del sistema.
- **Retención:** si el servidor purga frames, el historial intermedio se pierde y la app lo indica con `historyLimited`.
- **Reloj del teléfono:** la antigüedad y la «ingesta interrumpida» se calculan con la hora local; un reloj muy desfasado las desplaza.
- `appliedFrames` se emite pero ninguna pantalla lo consume todavía; las gráficas del historial consultan `framesBetween`.

12.4 Funcionalidad fuera de esta versión

Elemento	Estado	Referencia
Mapa asistido 2D (plano, ruta, habitaciones)	Retirado; leyenda «No disponible»	ADR-009, R11
Agregados métricos (`metric_aggregates`)	Puerto sin adaptador (`null`)	B9
Creación de misiones desde la app	«Guardar» no envía nada	R9
Calibraciones, CSV, push, video y comandos	No implementados	R5
Acuse de alertas (`ack_alert`)	No usado: la app es de solo lectura	RB25
Iconografía vectorizada de imágenes de 200 px	Bordes algo irregulares; reemplazar por los SVG originales	R12

12.5 Rendimiento

En el Galaxy A30 un recorrido por todas las pantallas deja ~92 MB de memoria total y arranque en frío de ~1.1 s con R8. Cerca de **una cuarta parte de los cuadros son lentos** en ese teléfono de gama baja (sin mejora con R8); la mitigación ya aplicada es reciclar objetos `Path` en la gráfica con `remember`. Antes de optimizar más, medir con `gfxinfo`.

13. Decisiones de diseño

Las decisiones están registradas como ADR en `docs/decisions/`. Esta tabla resume el **porqué** y lo que cuesta cada una.

Decisión	Motivo	Consecuencia / costo
Hexagonal en dos módulos (`:core` + `:app`) — ADR-001	Que el compilador impida que el dominio dependa de Android o Supabase; probar reglas en JVM sin emulador	Adaptadores y composición comparten módulo; si crecen, separar en `:adapters`
Inyección manual — ADR-002	Sin dependencias extra ni generación de código; toda la composición visible en un archivo	Hay que cablear a mano; un framework de DI se puede adoptar sin tocar `:core`
supabase-kt y DTO del contrato — ADR-003	Supabase es la persistencia común (Auth, RLS, Realtime); un solo SDK con tres módulos	Nombres remotos aislados en `SupabaseResources`; el esquema real cambió y obligó a `NormalizedFrameAssembler`
Parámetros iniciales configurables — ADR-004	Los valores del expediente (2 s, 3 periodos, 10 s, 30 s) son iniciales y deben ajustarse con datos reales	Cada política recibe su valor por constructor; no hay umbrales de gas en la app
Compose y solo backend real — ADR-005	Evitar simuladores que oculten problemas de contrato; mostrar «pendiente» donde no hay contrato	Sin backend válido no hay datos; la app muestra el error de configuración
Login con Supabase Auth — ADR-006	Autenticación formal con recuperación de acceso	`DevAccount` solo autocompleta en debug
Build `demo` de lectura pública — ADR-007	Poder mostrar datos reales sin un usuario con membresía	Temporal; depende de B19; se retira con B21
Identidad V.I.S.O.R. — ADR-008	Aplicar el manual de identidad corporativa	Íconos vectorizados desde PNG; conviene sustituirlos por SVG
Mapa 2D fuera de versión — ADR-009	Depende del plano que IoT aún no publica; evitar una función a medias	Se reintroduce puerto, caso de uso y vista cuando exista el contrato
El aviso no es el dato (RB26)	Realtime puede perder, duplicar o reordenar avisos	Cada aviso dispara una consulta por cursor; algo más de tráfico a cambio de corrección
Buffer de avisos acotado, `DROP_OLDEST`	Evitar crecimiento de memoria con ráfagas	Se pierden avisos, no datos; verificado con una ráfaga de 200
`ignoreUnknownKeys`	Los cambios aditivos del contrato v1 no deben romper la app	Un campo nuevo pasa inadvertido hasta que se modele
`null` ≠ 0, ADC ≠ ppm, silencio ≠ aire seguro	En un monitor de gases un falso «limpio» es peor que «sin dato»	Más estados en la interfaz (Sin dato, No instalado, Calentando…)
Frame rechazado avanza el cursor	Un frame defectuoso no debe bloquear el flujo	Hay que vigilar `rejectedFrames`
Sin base de datos local	Menos superficie de privacidad y de sincronización; el servidor es la fuente de verdad	Sin modo sin conexión; al reabrir se vuelve a consultar
Autorización solo en el servidor (RLS)	Ocultar botones no protege nada	`CurrentHomeScope` es solo una defensa de cliente

14. Seguridad y privacidad

Tema	Medida
Claves	Solo URL HTTPS y clave **pública**. `SupabaseConfig` rechaza `service_role` y `sb_secret_…`. Las claves viven en `local.properties` (ignorado por git) y se incrustan en `BuildConfig`; la clave publicable es pública por diseño
Credenciales	La contraseña no se guarda ni se registra. `DEV_EMAIL`/`DEV_PASSWORD` solo existen en debug y van vacíos en `release` y `demo`
Autorización	RLS en el servidor por membresía `active`; el cliente no decide permisos. `NotInScopeException` evita consultas fuera de ámbito
Datos en el teléfono	Solo tema y último correo (`SharedPreferences`). Tokens a cargo del SDK. `allowBackup=false`
Registros	No hay `Log`/`println` en el código propio; los mensajes de error nunca incluyen payloads ni cuerpos de respuesta
Transporte	HTTPS y WebSocket seguro; sin *pinning*. Permiso `INTERNET` únicamente
Repositorio público	Nunca se versionan claves, keystores (`*.jks`, `*.keystore`), APK (`*.apk`) ni `local.properties`. Los fixtures son sintéticos
Privacidad del domicilio	Solo se maneja un nombre de trabajo del domicilio, no la dirección postal
Modo demo (B19)	El APK `demo` lee con la clave pública: equivale a publicar la telemetría que el proyecto ya expone. Debe cerrarse en backend antes de un uso real

Antes de publicar un APK

Buscar `sb_secret_` y credenciales en el APK (sección 10.4), revisar que `DEV_*` estén vacíos y que ninguna captura o documento muestre correos o claves reales.

15. Guía de mantenimiento y extensión

15.1 Agregar un canal de telemetría

No hay una lista fija de canales: la app acepta cualquier `channel_key`. Un canal nuevo aparece en cuanto el robot lo publica y existe en `channels`.

1. Dar de alta el canal en `channels` (clave, `metric_type`, `unit_code`): lo hace backend.
2. Si necesita un título, ícono o agrupación propios, extender `ui/model/ChannelPresentation.kt`; sin eso se muestra con su clave.
3. Añadir un caso al fixture y a `ReadingInterpretationTest` si introduce una calidad o un tipo de valor nuevo.

15.2 Soportar una versión nueva del contrato (schema_version 2)

1. Crear un DTO y un mapeador para v2 junto a `TelemetryFrameMapper`; **no** modificar el de v1.
2. Elegir el mapeador por versión mayor en el adaptador (`toRecord`) y mantener `FrameRecord` como salida común.
3. Actualizar `NormalizedFrameAssembler` solo si cambia la forma del almacenamiento.
4. Agregar fixtures y pruebas de v2 y quitar `frame_qa_schema_v2.json` del caso de rechazo.

15.3 Cambiar el nombre de una tabla o columna remota

1. Actualizar la constante en `shell/supabase/SupabaseResources.kt` y, si es necesario, el DTO o la cadena `SELECT`.
2. Ejecutar `NormalizedFrameAssemblerTest` y las pruebas del mapeador; `:core` no debe tocarse.
3. Contrastar con el proyecto real de Supabase, no solo con el script SQL.

15.4 Agregar un puerto y su adaptador

Listado 48 · Plantilla de un puerto, su caso de uso y su adaptador

```
// :core  <dominio>/ports/XPort.kt
interface XPort { suspend fun load(robotId: RobotId): List<X> }

// :core  <dominio>/application/QueryX.kt  (valida el ámbito y delega)
class QueryX(private val port: XPort, private val scope: CurrentHomeScope) {
    suspend operator fun invoke(robotId: RobotId): List<X> {
        scope.requireRobot(robotId)
        return port.load(robotId)
    }
}

// :app  <dominio>/adapters/supabase/SupabaseXAdapter.kt
class SupabaseXAdapter(private val supabase: SupabaseClient) : XPort {
    override suspend fun load(robotId: RobotId) = supabaseCall("x") {
        supabase.from(SupabaseResources.X).select(PostgrestColumns.raw("id, ...")) {
```

```
            filter { eq(SupabaseResources.Columns.ROBOT_ID, robotId.value.toString()) }
        }.decodeList<XRowDto>().map { it.toDomain() }
    }
}
// shell/Adapters.kt: añadir el campo y construirlo en Adapters.supabase(...)
// shell/AppContainer.kt: instanciar QueryX con currentHomeScope y exponerlo
```

15.5 Agregar una pantalla

1. Crear `ui/<pantalla>/XScreen.kt` y `XViewModel.kt` (un único estado inmutable expuesto como `StateFlow`).
2. Obtener las dependencias con `uiDependencies` (no pasar `AppContainer` a los composables hijos).
3. Registrar la ruta (`@Serializable data object`) y su `composable<…>` en `MainScreen.kt`.
4. Usar solo tokens de `CentinelaTheme.colors` y probar con fuente 2.0 y ancho de 360 dp.

15.6 Publicar una versión

1. Subir `versionCode` y `versionName` en `app/build.gradle.kts` y escribir `docs/releases/vX.Y.Z.md`.
2. Ejecutar `:core:test` y `:app:testDebugUnitTest`; compilar `assembleRelease` y `assembleDemo`.
3. Verificar los APK (sección 10.4), probarlos en teléfono y calcular el SHA-256.
4. Con aprobación explícita: fusionar a `main`, crear la etiqueta y la *pre-release* con los APK, sus `.sha256` y el manual de usuario.

15.7 Problemas frecuentes de entorno

Síntoma	Causa probable	Solución
La app abre con «Falta supabase.url en local.properties»	Configuración ausente o vacía al compilar	Completar `local.properties` y recompilar (se incrusta en `BuildConfig`)
«supabase.publishableKey es una clave privilegiada»	Se pegó una `service_role` o `sb_secret_…`	Usar la clave publicable o `anon`
Inicia sesión pero no hay domicilios	El usuario no tiene membresía `active` (B21)	Pedir la alta a backend (`solicitud-usuario-demo.md`)
Todos los frames «no cumplen el contrato v1»	`device_sessions` no es legible (B20) o falta `schema_version`	Aplicar las políticas de lectura; en `demo` se usa `raw_payload`
Canales siempre «Atrasado»	La ingesta no publica frames nuevos	Revisar `received_at` del último frame; no es un fallo de la app
Sin avisos en vivo, solo cada 30 s	`telemetry_frames` no está en la publicación `supabase_realtime` (B4)	Pedir a backend `alter publication supabase_realtime add table …`
Compilación se detiene por memoria	Heap de Gradle (2 GB) y poca RAM	`--no-daemon --max-workers=1`, una tarea a la vez

16. Deuda técnica y pendientes

Pendientes que condicionan el trabajo técnico (el detalle completo está en `docs/pendientes.md`).

Código	Asunto	Impacto en el código
B19	El rol `anon` lee telemetría con la clave pública	Mantiene vivo el build `demo`; al cerrarse, la demo queda vacía
B21	No existe usuario con membresía activa	Impide probar el login válido; al resolverse se retira `demo`, `PublicRead*` y la rama `publicRead` del ensamblador
B20	`device_sessions` no legible sin políticas	La demo rodea el problema con `raw_payload`
B3	Snapshot consistente con marca de agua	Hoy es el frame más reciente; una RPC cambiaría `SupabaseTelemetryQueryAdapter.snapshot`
B9 · B16 · B8	Agregados, metadatos de misión y capacidades completas	Puertos parciales o sin adaptador
R9	«Generar nueva misión» no define contrato	`NewMissionSheet` no envía nada
R12	Íconos vectorizados desde PNG	Sustituir por SVG originales con `tools/identidad`
—	Firma de depuración y sin análisis estático	Definir keystore del equipo y agregar linters al pipeline
—	`appliedFrames` sin consumidores	Usarlo para gráficas en vivo o retirarlo
—	Sin pruebas de interfaz automatizadas	Valorar pruebas de Compose en dispositivo

Anexo A. Glosario

Término	Significado
Frame	Captura completa de todos los sensores en un instante (contrato v1.1)
`stream_version`	Cursor de commit por robot, asignado por el backend; ordena los frames persistidos
Snapshot / delta	Estado más reciente / frames posteriores al cursor
Watermark	Versión del flujo hasta la que un snapshot es consistente
Reconciliación	Consulta periódica por cursor que cubre avisos omitidos
RLS	*Row Level Security* de PostgreSQL: autorización por fila en el servidor
Ámbito (`HomeScope`)	Domicilio seleccionado y sus robots durante la sesión
Puerto / adaptador	Interfaz que el núcleo necesita / implementación concreta de infraestructura
Señal cruda	Valor de ADC de un sensor sin calibrar; no es una concentración
ADR	Registro de decisión de arquitectura (`docs/decisions/`)

Anexo B. Mapa de la documentación del repositorio

Documento	Contenido
`README.md`	Presentación, alcance y arranque rápido
`docs/architecture/`	Arquitectura hexagonal y sincronización de telemetría

Documento	Contenido
`docs/data_dictionary/frame-telemetria-v1.md`	Diccionario del frame (origen del capítulo 7)
`docs/backend/`	Esquema SQL del equipo, propuestas de políticas RLS y vista, datos de prueba, estado del proyecto Supabase
`docs/decisions/`	ADR-001 a ADR-009
`docs/ui/interfaz-movil.md`	Guía de interfaz y tokens
`docs/requirements/trazabilidad.md`	Trazabilidad entre requisitos y código
`docs/releases/`	Notas de cada versión
`docs/manual-de-usuario/`	Manual de usuario (Word y PDF)
`tools/identidad/`	Scripts de vectorización del isotipo y los íconos

Anexo C. Licencias de terceros

Poppins y Questrial se distribuyen bajo la **SIL Open Font License 1.1** (`app/OFL-Poppins.txt`, `app/OFL-Questrial.txt`). Jetpack, Kotlin, Ktor, kotlinx y supabase-kt se usan bajo licencia Apache 2.0. El repositorio incluye su propia `LICENSE`.